



Báo cáo cuối kì Kỹ thuật phần mềm nhóm 6 Quản lý tài nguyên thiên nhiên

Kỹ thuật phần mềm ứng dụng (Trường Đại học Bách khoa Hà Nội)



Scan to open on Studocu

ĐẠI HỌC BÁCH KHOA HÀ NỘI
TRƯỜNG ĐIỆN-ĐIỆN TỬ



Môn học: Kỹ thuật phần mềm ứng dụng

BÁO CÁO CUỐI KÌ

Đề tài: Hệ thống quản lý tài nguyên thiên nhiên

Nhóm 6:	Lê Văn Minh (nhóm trưởng)	20214001
	Cao Danh Khang	20213967
	Nguyễn Minh Hiếu	20210352
	Trương Đức Trường	20210880
	Nguyễn Như Huy	20213951

Mã lớp: 145539 – ET3260

Giảng viên hướng dẫn: Ths. Vũ Song Tùng

Hà Nội, 01/2024

MỤC LỤC

Lời nói đầu	5
CHƯƠNG 1: TỔNG QUAN	8
Giới thiệu:	8
1.1. Giới thiệu chung	8
1.2 Các hoạt động trong khuôn khổ đề tài	8
1.3 Những các làm hiện nay	8
1.4 Các công cụ được sử dụng trong đề tài	8
1.4.1. Mô hình client-server	8
1.4.2 Back-end: Java và Spring Framework	10
1.4.3. Front-end: HTML, CSS , Javascript và Vuejs	11
1.4.4 Cơ sở dữ liệu và hệ quản trị cơ sở dữ liệu	12
CHƯƠNG 2: PHÂN TÍCH THIẾT KẾ HỆ THỐNG	13
Giới thiệu:	13
2.1 DFD (Data Flow Diagram)	13
2.1.1. DFD mức ngữ cảnh	13
2.1.2. DFD mức đỉnh	14
2.1.3. DFD mức dưới đỉnh	15
2.2 FHD (Function Hierarchy Diagram)	18
2.3 ERD (Entity Relation Diagram)	22
2.4 Kết luận	23
CHƯƠNG 3: TRIỂN KHAI	24
Giới thiệu:	24
3.1 Thiết kế cơ sở dữ liệu	24
3.1.1. Các thực thể và thuộc tính	24
3.1.2. Từ điển dữ liệu pha phân tích:	25
3.1.3. Bảng database	26
3.1.4. Pha thiết kế	27
3.2 Phần code	33
3.2.1 Đơn vị hành chính - Administration	33

3.2.2 Người dùng – User.....	36
3.2.3 Cơ sở lưu trữ động vật – AnimalStorageFacilities	38
3.2.4 Tọa độ trên bản đồ - Coordinates DTO	40
3.2.5 Loài động vật – AnimalSpecies	40
3.2.6 Loại biến động – Fluctuation	41
3.2.7 Thống kê theo tháng/quý/năm	41
3.2.8 Cơ sở sản xuất giống cây trồng - PlantFacilities	47
3.2.9 Giống cây trồng	49
3.2.10 Cơ sở sản xuất chế biến gỗ - WoodFacilities	50
3.2.11 Loại hình sản xuất – ProductionType	51
3.2.12 Hình thức hoạt động – OperationForm.....	52
3.3 Giao diện trực quan.....	53
3.3.1 Giao diện đăng nhập	53
3.3.2 Giao diện chính.....	54
3.3.3 Giao diện quản lý động vật.....	55
3.3.4 Giao diện quản lý giống cây trồng.....	57
3.3.5 Giao diện quản lý cơ sở sản xuất gỗ	58
3.3.6 Giao diện Bản đồ	60
3.3.7 Quản lý hệ thống.....	61
3.4 Kết luận chung	62
Kết luận	63

HÌNH ẢNH

Hình 1.1 Mô hình Client - Server	9
Hình 2.1 DFD mức ngữ cảnh.....	13
Hình 2.2 DFD mức đỉnh	14
Hình 2.3 DFD quản lí hành chính.....	15
Hình 2.4 DFD về quản lý động vật.....	15
Hình 2.5 DFD quản lý giống cây trồng	16
Hình 2.6 DFD quản lý cơ sở sản xuất.....	16
Hình 2.8 DFD quản lý hệ thống.....	17
Hình 2.7 DFD quản lý hệ thống.....	17
Hình 2.9 Sơ đồ quản trị hệ thống.....	18
Hình 2.10 Sơ đồ chức năng phân quyền.....	18
Hình 2.11 Sơ đồ Quản lí lịch sử.....	19
Hình 2.12 Sơ đồ Quản lí người dùng.....	19
Hình 2.13 Sơ đồ Báo cáo thống kê	20
Hình 2.14 Sơ đồ quản lí tài khoản	20
Hình 2.15 Sơ đồ quản lí tài nguyên	21
Hình 2.16 Sơ đồ quản lí sản xuất gỗ.....	21
Hình 2.17 Sơ đồ quản lí cây giống.....	22
Hình 2.18 Sơ đồ quản lí lưu trữ động vật	22
Hình 2.19 Sơ đồ ERD	23
Hình 3.1 Bảng thiết kế database	26
Hình 3.2 Giao diện đổi mật khẩu.....	53
Hình 3.3 Giao diện đăng nhập	53
Hình 3.4 Trang chủ, logo dự án	54
Hình 3.5 Quản lý hành chính	54
Hình 3.6 Giao diện các loại động vật.....	55
Hình 3.7 Giao diện cơ sở lưu trữ động vật	55
Hình 3.8 Giao diện biểu đồ thống kê tổng số lượng động vật tại các cơ sở.....	56
Hình 3.9 Giao diện các Loại cây trồng.....	57
Hình 3.10 Giao diện cơ sở sản xuất giống cây trồng.....	57
Hình 3.11 Giao diện các Loại hình sản xuất gỗ.....	58
Hình 3.12 Giao diện các Hình thức hoạt động cơ sở sản xuất gỗ.....	58
Hình 3.13 Giao diện các Cơ sở sản xuất gỗ.....	59
Hình 3.14 Giao diện Biểu đồ tổng số lượng gỗ lưu trữ tại các cơ sở.....	59
Hình 3.15 Giao diện Bản đồ phân bố của các cơ sở.....	60
Hình 3.16 Giao diện tài khoản người dùng.....	61
Hình 3.17 Giao diện quản lý Users.....	61

BẢNG BIỂU

Bảng 3.1 Các thực thể và thuộc tính	24
Bảng 3.2 Bảng phân tích	25
Bảng 3.4 Các thuộc tính bảng administrative_levels (Cấp hành chính)	27
Bảng 3.5 Các thuộc tính bảng administrations (Hành chính)	27
Bảng 3.6 Các thuộc tính bảng roles (Vai trò)	27
Bảng 3.7 Các thuộc tính bảng users (người dùng)	28
Bảng 3.8 Các thuộc tính bảng facility_types (loại cơ sở)	28
Bảng 3.9 Các thuộc tính bảng facilities (cơ sở)	29
Bảng 3.10 Các thuộc tính bảng operation_types (hình thức hoạt động)	29
Bảng 3.11 Các thuộc tính bảng coordinates (tọa độ)	30
Bảng 3.12 Các thuộc tính bảng fluctuation (loại biến động)	30
Bảng 3.13 Các thuộc tính bảng animal_species (loài động vật)	30
Bảng 3.14 Các thuộc tính bảng production_type (loại hình sản xuất)	31
Bảng 3.15 Các thuộc tính bảng plant_seed (giống cây trồng)	31
Bảng 3.16 Các thuộc tính bảng statistics (số liệu thống kê)	32
Bảng 3.17 Các thuộc tính bảng statistics (số liệu thống kê)	32
Bảng 3.18 Các thuộc tính access_history (lịch sử truy cập)	33

Lời Nói Đầu

Những năm gần đây, vấn đề môi trường, bảo vệ tài nguyên thiên nhiên đang được chính phủ Việt Nam quan tâm hơn bao giờ hết. Nạn chặt phá cây rừng để lấy gỗ, săn bắt động vật trái phép để buôn bán vẫn đang là vấn đề đáng báo động. Nó gây ra những hệ lụy tiêu cực như biến đổi khí hậu, phá hủy hệ sinh thái, cắt đứt chuỗi thực ăn, ... và những hệ lụy kéo theo như nước biển dâng, thiên tai; cuộc sống con người bị đe dọa cả về điều kiện sống và tính mạng. Ngoài ra, như đã biết, chính phủ Việt Nam đang quyết tâm thực hiện mục tiêu Net zero (đưa mức phát thải ròng về 0) thì việc bảo vệ tài nguyên thiên nhiên như rừng, động vật lại càng trở lên quan trọng hơn bao giờ hết. Vì vậy, bài toán đặt ra là phải có những cách quản lý tài nguyên thiên nhiên. Từ đó, có những biện pháp kịp thời để bảo vệ và phát triển tài nguyên thiên nhiên nước nhà.

Hiện nay, khoa học công nghệ phát triển nhanh như vũ bão, đặc biệt là lĩnh vực công nghệ thông tin. Có thể nói, công nghệ và chuyển đổi số đã được ứng dụng vào hầu hết tất cả mọi lĩnh vực trong cuộc sống. Trong đó, công nghệ phần mềm luôn đóng vai trò tiên phong, dẫn đầu trong quá trình đó. Các phần mềm đã, đang và sẽ được ứng dụng ngày càng nhiều trong công việc cũng như cuộc sống thường ngày của con người.

Phần mềm này được nhóm 6 xây dựng nhằm hỗ trợ cho chính phủ Việt Nam cũng như từng người quản lý được tài nguyên thiên nhiên, tài sản quốc gia bao gồm: Động vật, cây trồng, cơ sở sản xuất chế biến gỗ. Dựa vào phần mềm, các cấp quản lý, những hộ gia đình, những cá thể có thể truy cập thông tin dễ dàng, có cái nhìn trực quan và quản lý dễ dàng tài nguyên thiên nhiên. Từ đó, đề ra được những chính sách đúng đắn, những hành động kịp thời để bảo vệ, ngăn chặn kịp thời những ảnh hưởng tiêu cực và phát triển tài nguyên của quốc gia.

Đây là phần mềm đầu tiên mà nhóm 6 – Nature Care thực hiện nên không thể tránh khỏi những hạn chế và thiếu sót, kính mong giảng viên và các bạn bổ sung, góp ý để phần mềm ngày càng hoàn thiện và thực sự hữu ích với người sử dụng.

Nhóm 6 xin chân thành cảm ơn !

LỜI CAM ĐOAN

Chúng tôi xin cam đoan báo cáo “**Phần mềm quản lý tài nguyên thiên nhiên**” là công trình nghiên cứu của chính chúng tôi. Những phần có sử dụng tài liệu tham khảo có trong báo cáo đã được liệt kê và nêu rõ ra tại phần tài liệu tham khảo. Đồng thời những số liệu hay kết quả trình bày trong đồ án đều mang tính chất trung thực, không sao chép, đạo nhái.

Nếu như sai, chúng tôi xin hoàn toàn chịu trách nhiệm với giảng viên bộ môn và nhà trường.

CHƯƠNG 1: TỔNG QUAN

Giới thiệu:

Ở chương 1, đề cập đến các nội dung bao gồm nêu ra bài toán quản lý tài nguyên, đưa ra hướng giải quyết là sử dụng công nghệ, tạo ra một phần mềm với các tính năng phù hợp. Tiếp theo là những cái mà xã hội đã và đang làm, nhược điểm của các phương pháp quản lý truyền thống và đưa ra những ưu điểm của sử dụng công nghệ phần mềm. Tiếp đến là những công cụ hỗ trợ xây dựng phần mềm được sử dụng (JavaScript, HTML, Springboot,...); những mô hình áp dụng cho bài toán (client-server); biểu đồ trực quan hóa bài toán, dữ liệu.

1.1. Giới thiệu chung

Bài toán sử dụng phần mềm quản lý → quản lý tài nguyên như cs dx giống cây trồng, cơ sở sản xuất gỗ, cơ sở lưu trữ động vật → kiểm soát số lượng, thống kê, so sánh,....

1.2 Các hoạt động trong khuôn khổ đề tài

- Quản lý các cấp hành chính
- Quản lý cơ sở sản xuất giống cây trồng
- Quản lý cơ sở sản xuất gỗ
- Quản lý cơ sở lưu trữ động vật

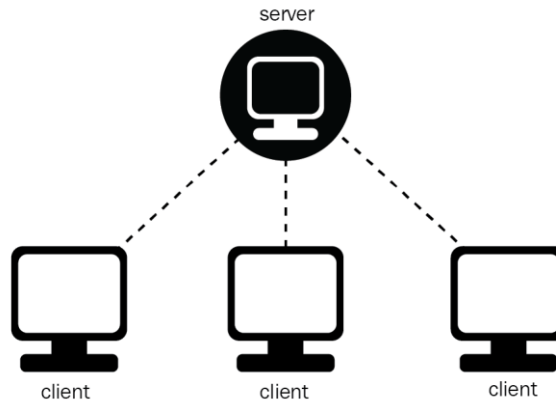
1.3 Những cái làm hiện nay

- Những phần mềm quản lý hiện nay: quản lý bán hàng, quản lý sổ tay chỉ tiêu,...
- Những cái cách mà người ta dùng để quản lý gỗ, động vật, cây

1.4 Các công cụ được sử dụng trong đề tài

1.4.1. Mô hình client-server [1]

Mô hình Client Server là mô hình mạng máy tính trong đó các máy tính con được đóng vai trò như một máy khách, chúng làm nhiệm vụ gửi yêu cầu đến các máy chủ. Để máy chủ xử lý yêu cầu và trả kết quả về cho máy khách đó.



Hình 1.1 Mô hình Client - Server

Trong mô hình Client - Server, server chấp nhận tất cả các yêu cầu hợp lệ từ mọi nơi khác nhau trên Internet, sau đó trả kết quả về máy tính đã gửi yêu cầu đó

Máy tính được coi là máy khách khi chúng làm nhiệm vụ gửi yêu cầu đến các máy chủ và đợi câu trả lời được gửi về. Để máy khách và máy chủ có thể giao tiếp được với nhau thì giữa chúng phải có một chuẩn nhất định, và chuẩn đó được gọi là giao thức. Một số giao thức được sử dụng phổ biến hiện nay như: HTTPS, TCP/IP, FTP,...

Nếu máy khách muốn truy xuất các thông tin từ máy chủ, chúng phải tuân theo một giao thức mà máy chủ đó đưa ra. Nếu yêu cầu đó được chấp nhận thì máy chủ sẽ thu thập thông tin và trả về kết quả cho máy khách yêu cầu. Bởi vì Server - máy chủ luôn luôn trong trạng thái sẵn sàng để nhận request từ client nên chỉ cần client gửi yêu cầu tín hiệu và chấp nhận yêu cầu đó thì server sẽ trả kết quả về phía client trong thời gian ngắn nhất.

Ưu điểm của mô hình:

- Giúp chúng ta có thể làm việc trên bất kì một máy tính nào có hỗ trợ giao thức truyền thông. Giao thức chuẩn này cũng giúp các nhà sản xuất tích hợp lên nhiều sản phẩm khác nhau mà không gặp phải khó khăn gì.
- Có thể có nhiều server cùng làm một dịch vụ, chúng có thể nằm trên nhiều máy tính hoặc một máy tính.
- Chỉ mang đặc điểm của phần mềm mà không hề liên quan đến phần cứng, ngoài yêu cầu duy nhất là server phải có cấu hình cao hơn các client.
- Hỗ trợ người dùng nhiều dịch vụ đa dạng và sự tiện dụng bởi khả năng truy cập từ xa.
- Cung cấp một nền tảng lý tưởng, cho phép cung cấp tích hợp các kỹ thuật hiện đại như mô hình thiết kế hướng đối tượng, hệ chuyên gia, hệ thông tin địa lý (GIS).

Nhược điểm:

- Vấn đề bảo mật dữ liệu thông tin đôi khi còn chưa được an toàn do phải trao đổi dữ liệu giữa 2 máy tính khác nhau ở 2 khu vực địa lý cách xa nhau. Và đây cũng nhược điểm duy nhất của mô hình này. Tuy nhiên vấn đề này có một số giao thức đã hỗ trợ bảo mật dữ liệu khi truyền tải. Giao thức được sử dụng phổ biến như HTTPS.

FrontEnd và BackEnd:

Từ mô hình Client – Server, giới lập trình viên hiện đại đã phân trang web ra làm hai phần để thuận tiện phát triển và quản lý: Front-end - Back-end

Phần front-end của một trang web là phần tương tác với người dùng. Tất cả mọi thứ ta nhìn thấy khi điều hướng trên Internet, từ các font chữ, màu sắc cho tới các menu xổ xuống và các thanh trượt, là một sự kết hợp của HTML, CSS, và JavaScript được điều khiển bởi trình duyệt máy tính của ta.

Đâu là những phần giúp phần front-end của một trang web có thể hoạt động được? Tất cả dữ liệu sẽ được lưu trữ ở đâu? Đó là phần việc của back end. Phần back end của một trang web bao gồm một máy chủ, một ứng dụng, và một cơ sở dữ liệu. Một lập trình viên back-end xây dựng và duy trì công nghệ mà sức mạnh của những thành phần đó, cho phép phần giao diện người dùng của trang web có thể tồn tại được.

1.4.2 Back-end: Java và Spring Framework

1.4.2.1. Java [2]

Java được phát triển bởi một nhóm do James Gosling lãnh đạo tại Sun Microsystems. Sun Microsystems đã được Oracle mua lại vào năm 2010. Ban đầu có tên là Oak, Java được thiết kế vào năm 1991 để sử dụng trong chip nhúng cho các thiết bị điện tử. Vào năm 1995, sau khi đổi tên thành Java, nó đã được thiết kế lại để phát triển ứng dụng Web. Java đã trở nên cực kỳ phổ biến. Sự tăng trưởng nhanh chóng và bao phủ rộng rãi của nó đến từ các đặc tính thiết kế. Java đơn giản, hướng đối tượng, phân tán, diễn giải, mạnh mẽ, an toàn, không phụ thuộc kiến trúc, di động, hiệu suất cao, đa luồng và linh hoạt, đặc biệt Java được tạo ra với tiêu chí “*Viết (code) một lần, thực thi khắp nơi*” (Write Once, Run Anywhere – WORA).

Java là một ngôn ngữ lập trình đa chức năng, tổng quát, được sử dụng để phát triển các ứng dụng quan trọng. Ngày nay, nó không chỉ được sử dụng cho lập trình Web mà còn để phát triển các ứng dụng độc lập hoạt động trên nhiều nền tảng trên máy chủ, máy tính để bàn và thiết bị di động.

1.4.2.2. Spring Framework [3]

Spring Framework được phát triển vào năm 2003 bởi Rod Johnson. Đây là một trong những mã nguồn mở được xây dựng dựa trên ngôn ngữ lập trình Java. Đến nay, Spring được hàng triệu lập trình viên lựa chọn để sử dụng. Nhờ nó mà các lập trình viên có thể phát triển ứng dụng Java một cách dễ dàng với tốc độ cao đáng kinh ngạc.

Spring khá nhẹ (version chỉ khoảng 2MB với kích thước nhỏ) và trong suốt (hoạt động trong suốt với lập trình viên). Spring cũng cung cấp và hỗ trợ các cơ sở hạ tầng đã được xác định rõ ràng để phát triển ứng dụng trong Java. Nói dễ hiểu hơn, Spring xử lý cơ sở hạ tầng để bạn tập trung tối đa vào việc phát triển ứng dụng.

Những tính năng cốt lõi của Spring phù hợp để sử dụng ở nhiều mảng khác nhau. Nó có thể hỗ trợ tốt từ các ứng dụng mobile, desktop đến các ứng dụng Java web.

1.4.3. Front-end: HTML, CSS , Javascript và Vuejs

1.4.3.1. HTML [4]

HTML là viết tắt của cụm từ Hypertext Markup Language (tạm dịch là Ngôn ngữ đánh dấu siêu văn bản). HTML được sử dụng để tạo và cấu trúc các thành phần trong trang web hoặc ứng dụng, phân chia các đoạn văn, heading, titles, blockquotes...

và HTML không phải là ngôn ngữ lập trình.

Một tài liệu HTML được hình thành bởi các phân tử HTML (HTML Elements) được quy định bằng các cặp thẻ (tag và attributes). Các cặp thẻ này được bao bọc bởi một dấu ngoặc nhọn (ví dụ <html>) và thường là sẽ được khai báo thành một cặp, bao gồm thẻ mở và thẻ đóng. Ví dụ, chúng ta có thể tạo một đoạn văn bằng cách đặt văn bản vào trong cặp tag mở và đóng văn bản <p> và </p> :

<p>Đây là cách bạn thêm đoạn văn trong HTML.</p>

Nhưng một số thẻ đặc biệt lại không có thẻ đóng và dữ liệu được khai báo sẽ nằm trong các thuộc tính (ví dụ như thẻ).

Cha đẻ của HTML là Tim Berners-Lee, cũng là người khai sinh ra World Wide Web và chủ tịch của World Wide Web Consortium (W3C – tổ chức thiết lập ra các chuẩn trên môi trường Internet). Các thiết lập và cấu trúc HTML được vận hành và phát triển bởi World Wide Web Consortium (W3C).

1.4.3.2. CSS [5]

CSS là chữ viết tắt của Cascading Style Sheets, nó là một ngôn ngữ được sử dụng để tìm và định dạng lại các phân tử được tạo ra bởi các ngôn ngữ đánh dấu (HTML). Nói ngắn gọn hơn là ngôn ngữ tạo phong cách cho trang web. Bạn có thể hiểu đơn giản rằng, nếu HTML đóng vai trò định dạng các phân tử trên website như việc tạo ra các đoạn văn bản,

các tiêu đề, bảng,...thì CSS sẽ giúp chúng ta có thể thêm style vào các phần tử HTML đó như đổi bố cục, màu sắc trang, đổi màu chữ, font chữ, thay đổi cấu trúc...

CSS được phát triển bởi W3C (World Wide Web Consortium) vào năm 1996, vì HTML không được thiết kế để gắn tag để giúp định dạng trang web.

Phương thức hoạt động của CSS là nó sẽ tìm dựa vào các vùng chọn, vùng chọn có thể là tên một thẻ HTML, tên một ID, class hay nhiều kiểu khác. Sau đó là nó sẽ áp dụng các thuộc tính cần thay đổi lên vùng chọn đó.

Mối tương quan giữa HTML và CSS rất mật thiết. HTML là ngôn ngữ markup (nền tảng của site) và CSS định hình phong cách (tất cả những gì tạo nên giao diện website), chúng là không thể tách rời.

1.4.3.4. Vuejs [6]

Gọi tắt là Vue (phát âm là /vju:/, giống như view trong tiếng Anh), Vue.js là một framework linh động dùng để xây dựng giao diện người dùng (user interfaces). Khác với các framework nguyên khối (monolithic), Vue được thiết kế từ đầu theo hướng cho phép và khuyến khích việc phát triển ứng dụng theo từng bước. Khi phát triển lớp giao diện (view layer), người dùng chỉ cần dùng thư viện lõi (core library) của Vue, vốn rất dễ học và tích hợp với các thư viện hoặc dự án có sẵn. Cùng lúc đó, nếu kết hợp với những kỹ thuật hiện đại như SFC (single file components) và các thư viện hỗ trợ, Vue cũng đáp ứng được dễ dàng nhu cầu xây dựng những ứng dụng một trang (SPA - Single-Page Applications) với độ phức tạp cao hơn nhiều.

1.4.4 Cơ sở dữ liệu và hệ quản trị cơ sở dữ liệu [7]

Cơ sở dữ liệu có tên tiếng anh là Database, viết tắt là CSDL. Là một tập hợp các dữ liệu có tổ chức, được lưu trữ và truy cập từ hệ thống máy tính. Khi cơ sở dữ liệu phức tạp hơn, chúng sẽ được phát triển bằng cách sử dụng các thiết kế và mô hình hóa hình thức. Tóm lại, cơ sở dữ liệu là hệ thống gồm rất nhiều thông tin, dữ liệu được xây dựng theo một cấu trúc nhất định để đáp ứng nhu cầu khai thác, sử dụng của nhiều người hay chạy nhiều chương trình cùng một lúc. Khi áp dụng hình thức lưu trữ này, sẽ khắc phục được những điểm yếu của việc lưu file thông thường trên máy tính. Các thông tin dữ liệu đảm bảo được sự nhất quán, hạn chế được việc trùng lặp thông tin.

Từ SQL là viết tắt của cụm Structured Query Language, có nghĩa là loại ngôn ngữ truy vấn có cấu trúc. Đây là một loại ngôn ngữ máy tính giúp hỗ trợ các thao tác lưu trữ, truy xuất dữ liệu cùng một cơ sở dữ liệu quan hệ một cách nhanh chóng. Ngôn ngữ này đã được ANSI – viện tiêu chuẩn quốc gia của Hoa Kỳ đặt tiêu chuẩn hóa, và trở thành một ngôn ngữ được sử dụng phổ biến.

Hệ quản trị cơ sở dữ liệu MySQL được sử dụng trong dự án này.

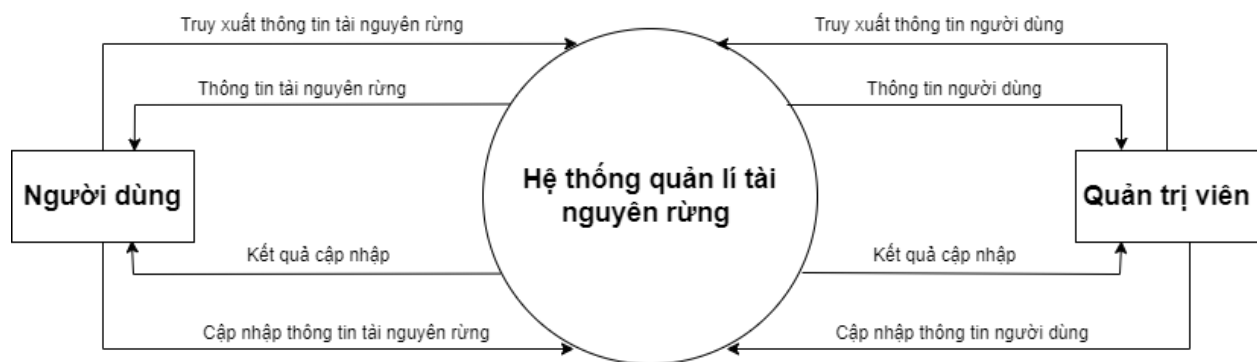
CHƯƠNG 2: PHÂN TÍCH THIẾT KẾ HỆ THỐNG

Giới thiệu:

Trong phần này, trình bày rõ ràng các bước từ thiết kế, phân tích hệ thống. Các nội dung gồm có sơ đồ DFD (DFD mức 1, DFD mức dưới đỉnh, DFD mức đỉnh, DFD mức ngữ cảnh), sơ đồ FHD, sơ đồ ERD.

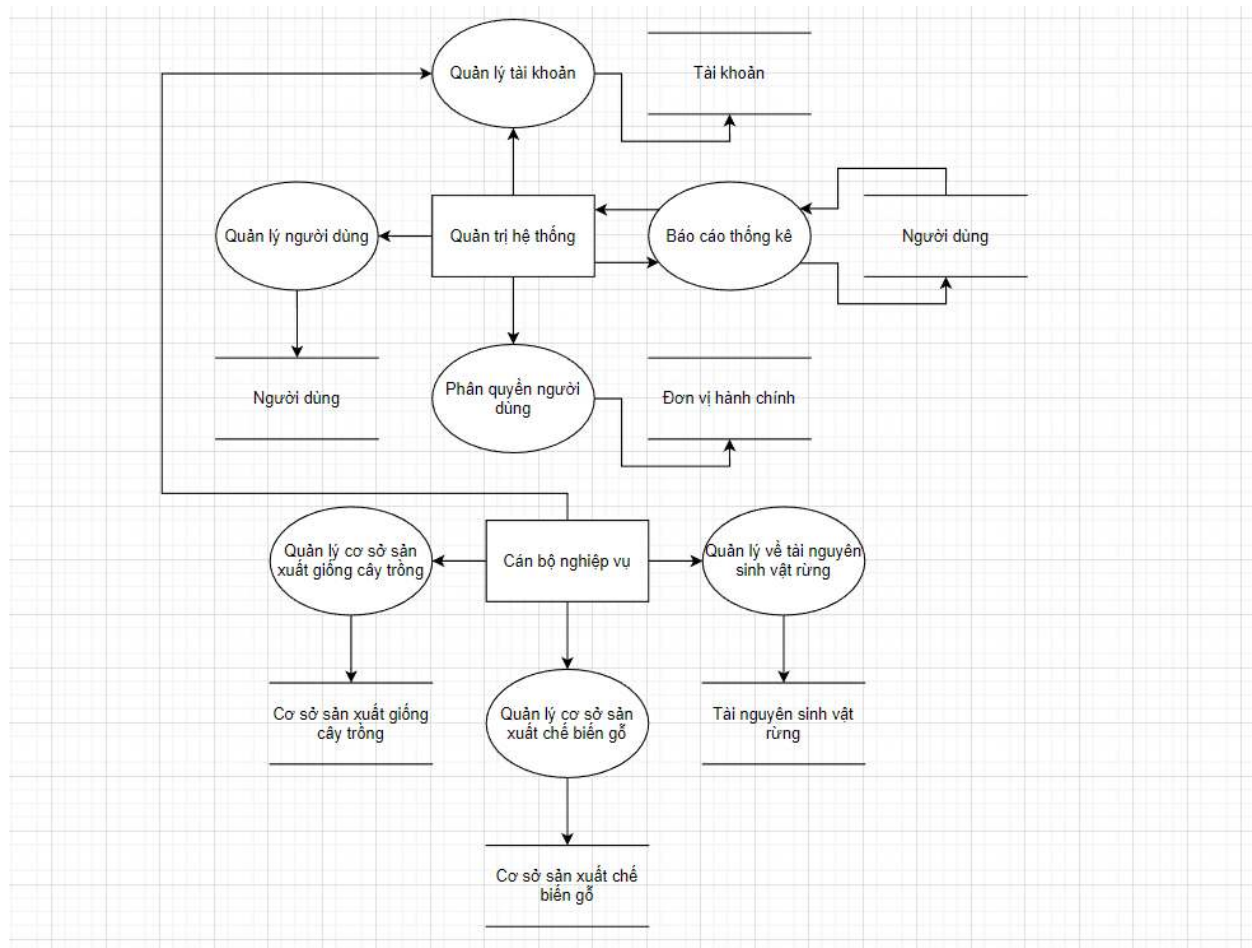
2.1 DFD

2.1.1. DFD mức ngữ cảnh



Hình 2.1 DFD mức ngữ cảnh

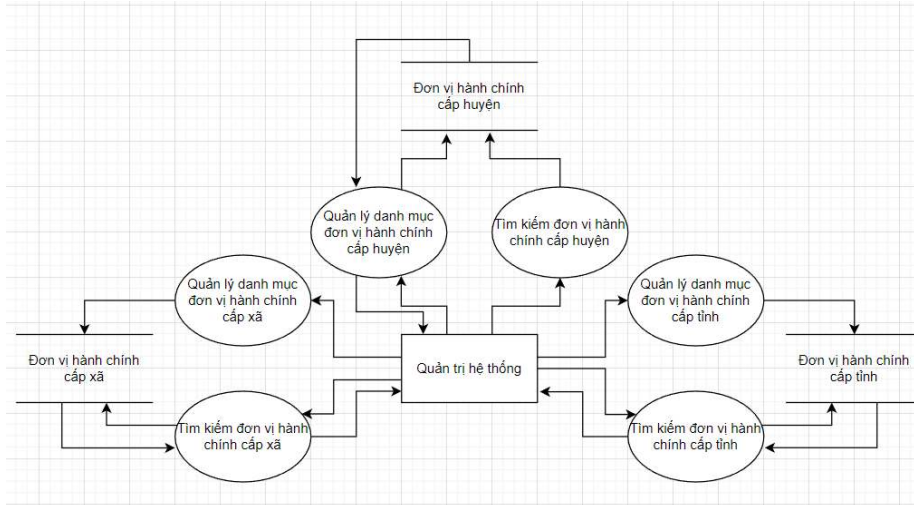
2.1.2. DFD mức đỉnh



Hình 2.2 DFD mức đỉnh

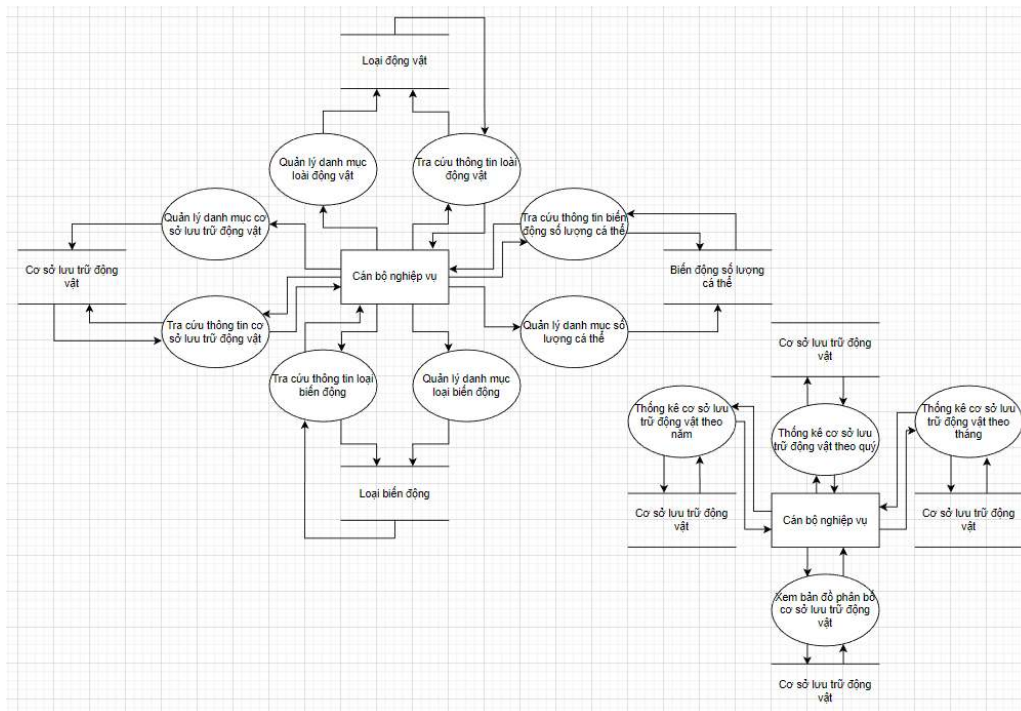
2.1.3. DFD mức dưới đỉnh

2.1.3.1 Chức năng 1: Quản lý hành chính



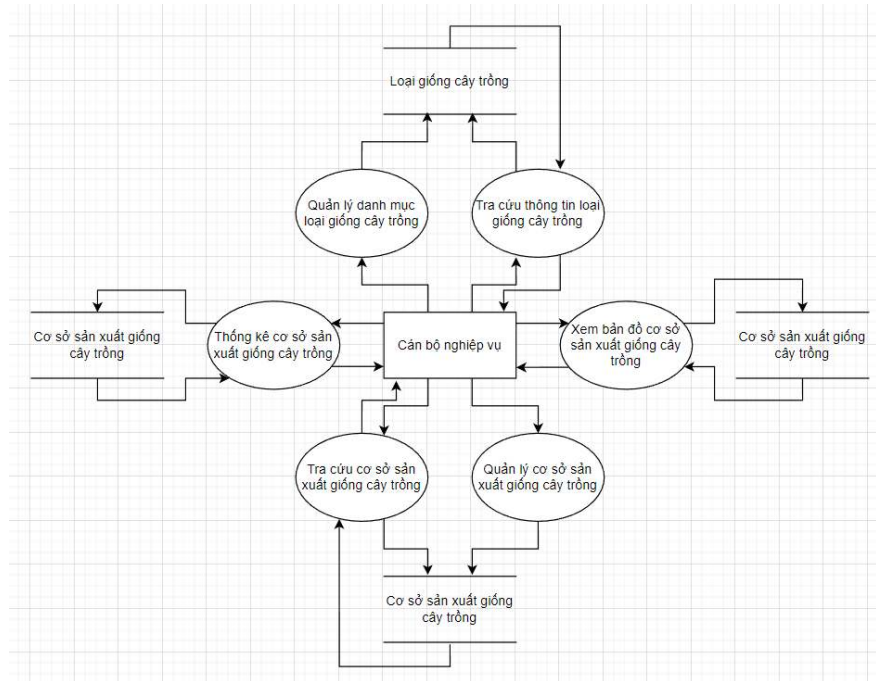
Hình 2.3 DFD quản lý hành chính

2.1.3.2 Chức năng 2: Quản lý động vật



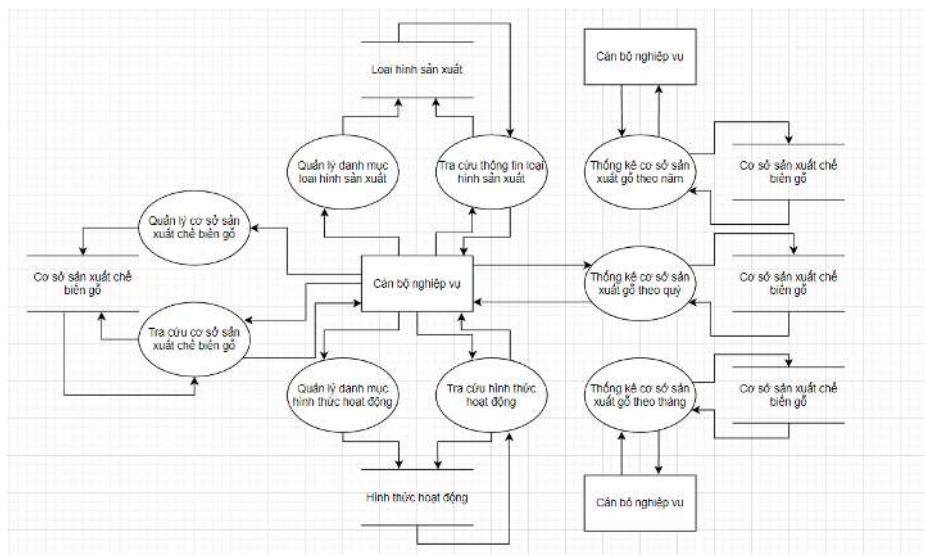
Hình 2.4 DFD về quản lý động vật

2.1.3.3 Chức năng 3: Quản lý giống cây trồng



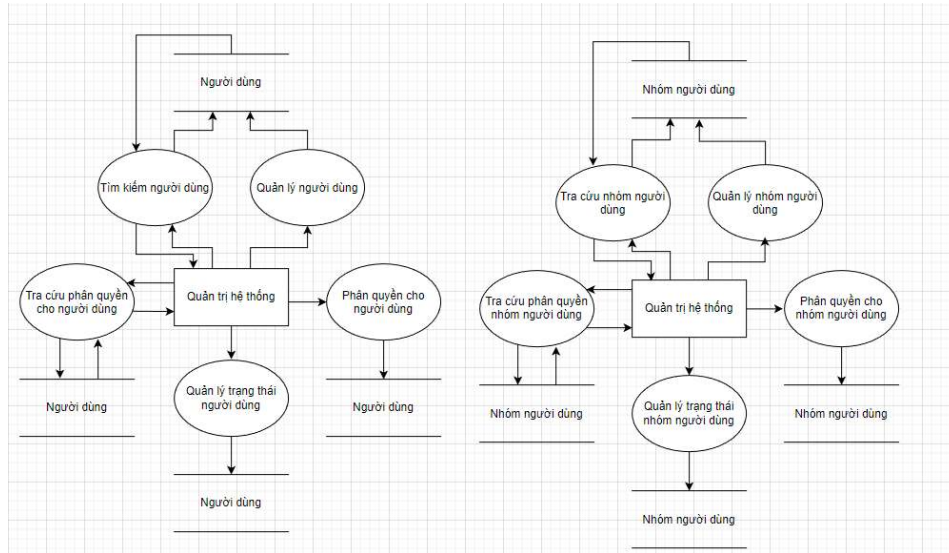
Hình 2.5 DFD quản lý giống cây trồng

2.1.3.4 Chức năng 4: Quản lý cơ sở sản xuất

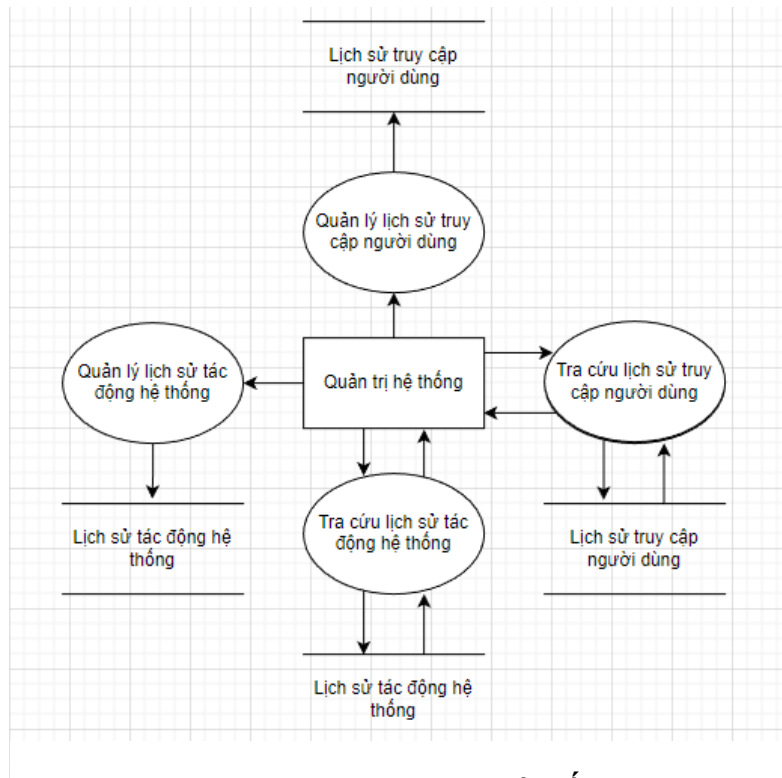


Hình 2.6 DFD quản lý cơ sở sản xuất

2.1.3.5 Chức năng 5: Quản lý hệ thống

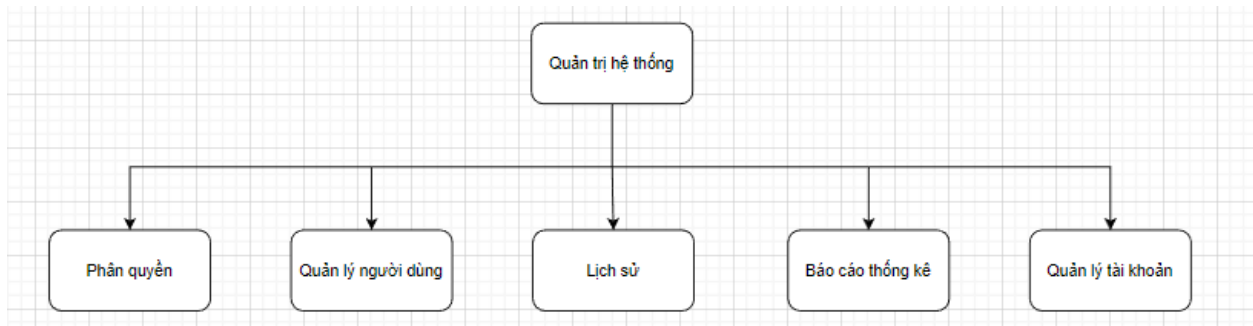


Hình 2.7 DFD quản lý hệ thống

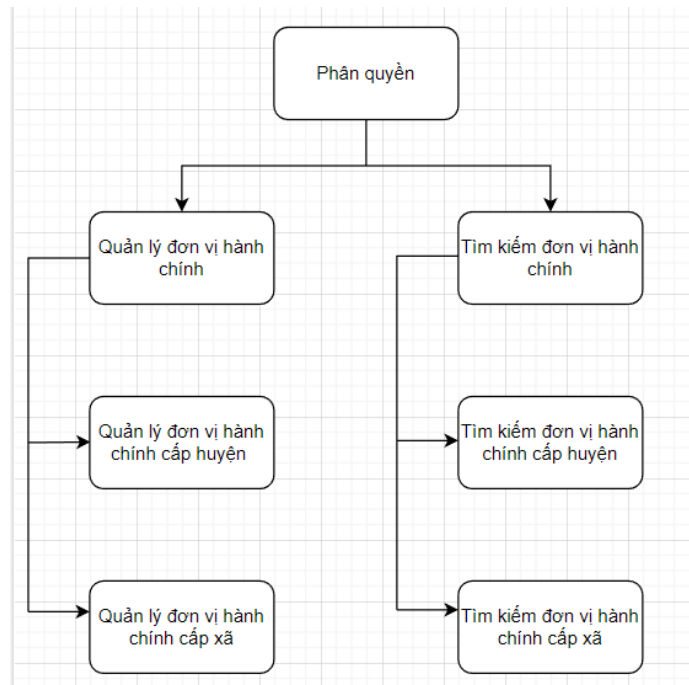


Hình 2.8 DFD quản lý hệ thống

2.2 FHD



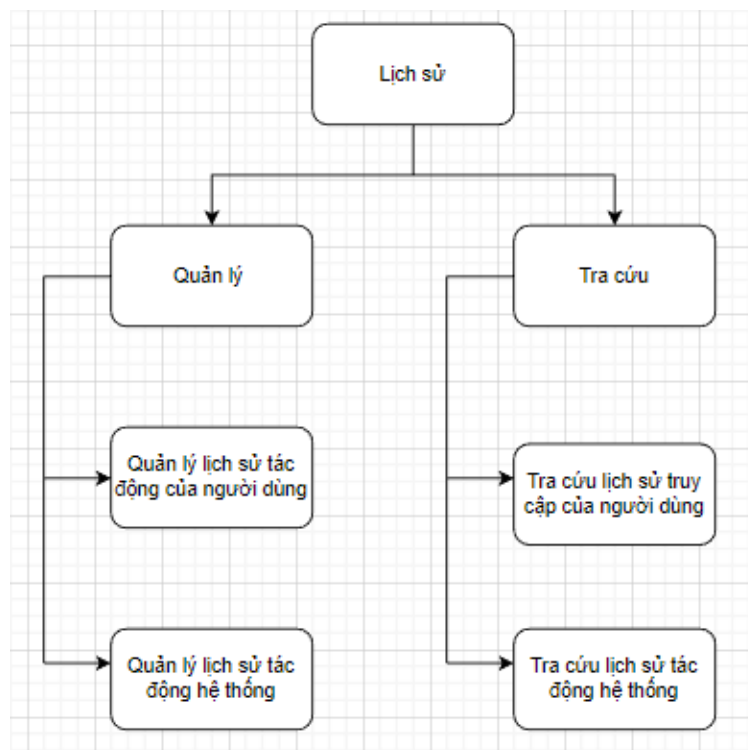
Hình 2.5 Sơ đồ quản trị hệ thống



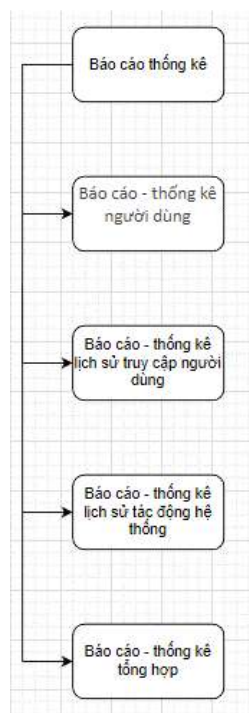
Hình 2.6 Sơ đồ chức năng phân quyền



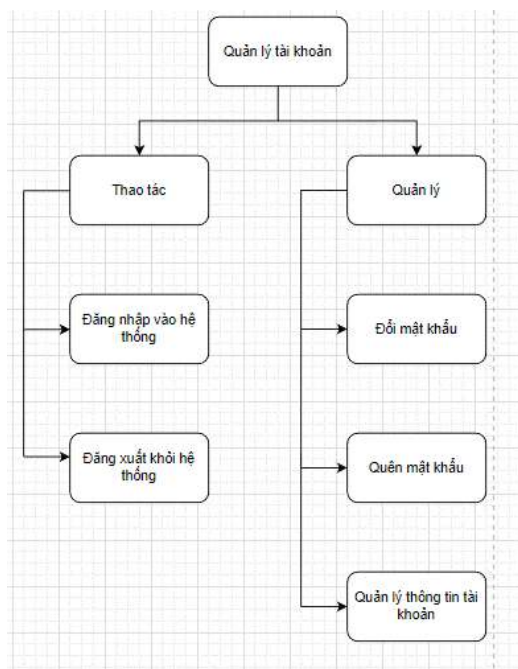
Hình 2.11. Sơ đồ Quản lý người dùng



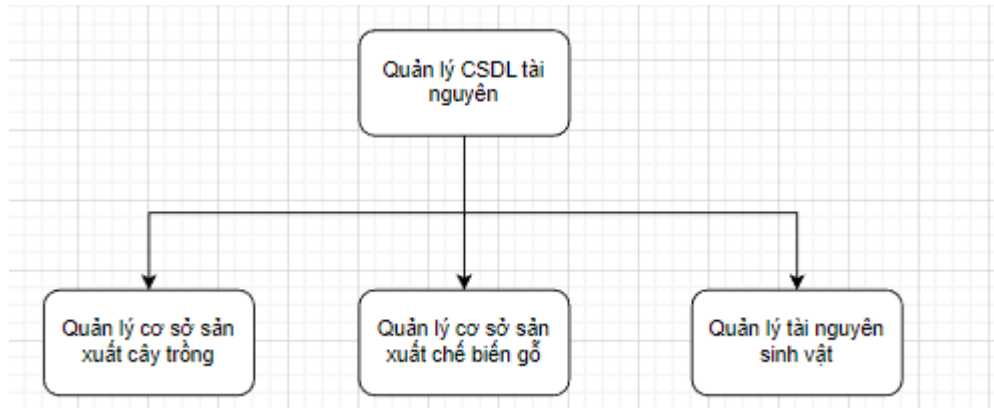
Hình 2.12. Sơ đồ Quản lý lịch sử



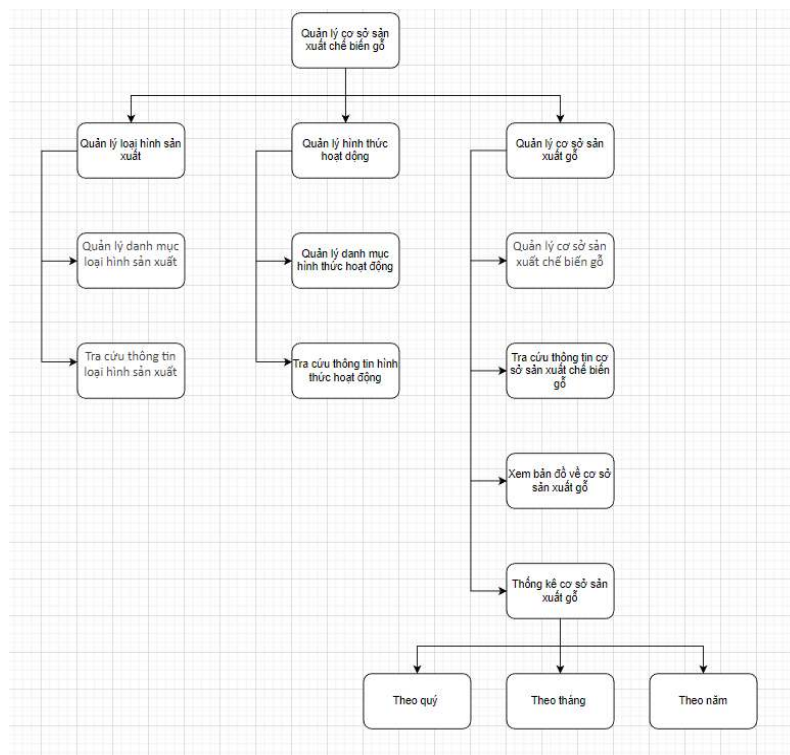
Hình 2.7: Sơ đồ Báo cáo thống kê



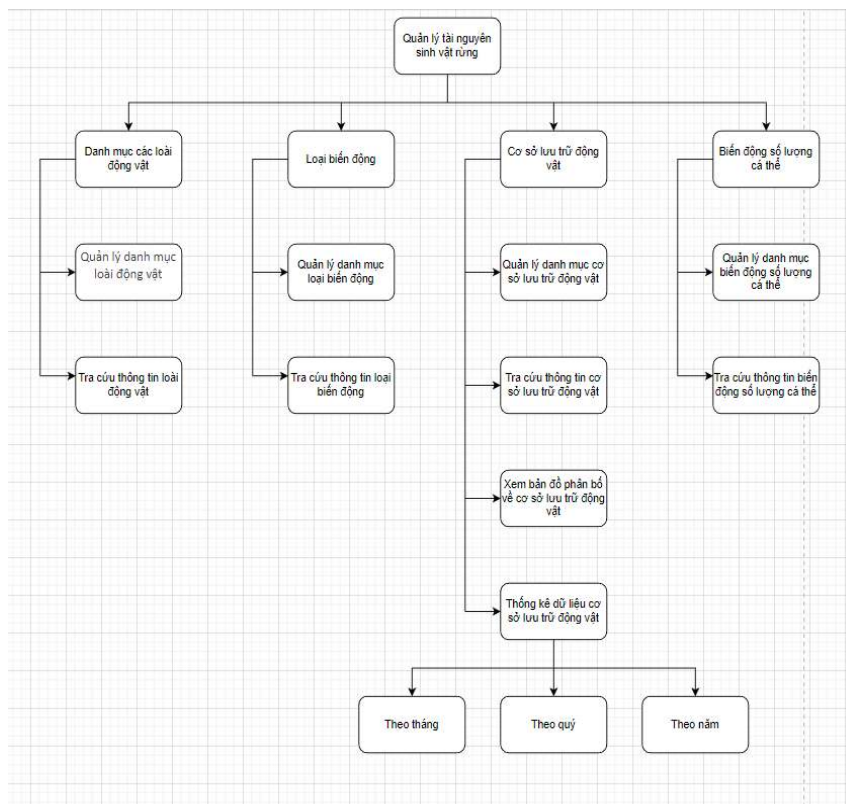
Hình 2.8: Sơ đồ quản lý tài khoản



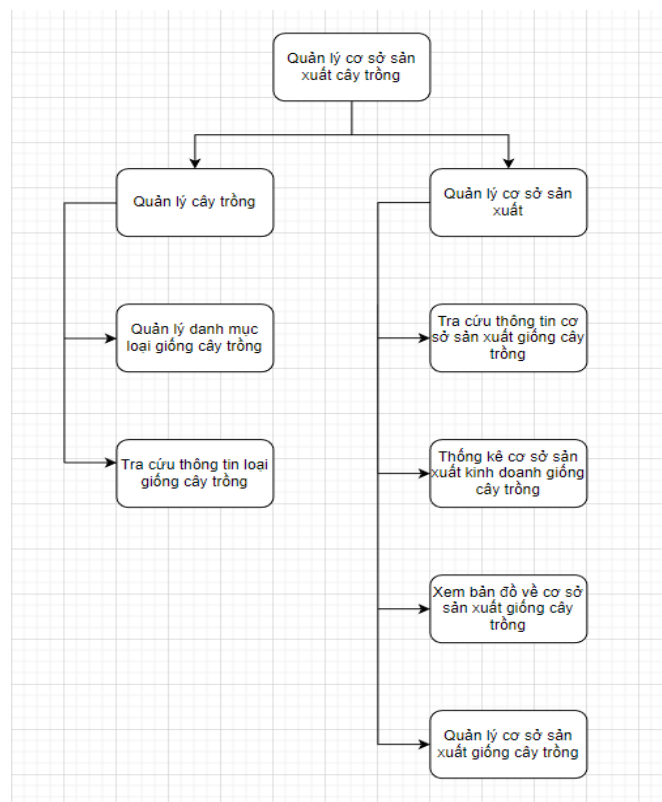
Hình 2.9: Sơ đồ quản lý tài nguyên



Hình 2.10: Sơ đồ quản lý sản xuất gỗ

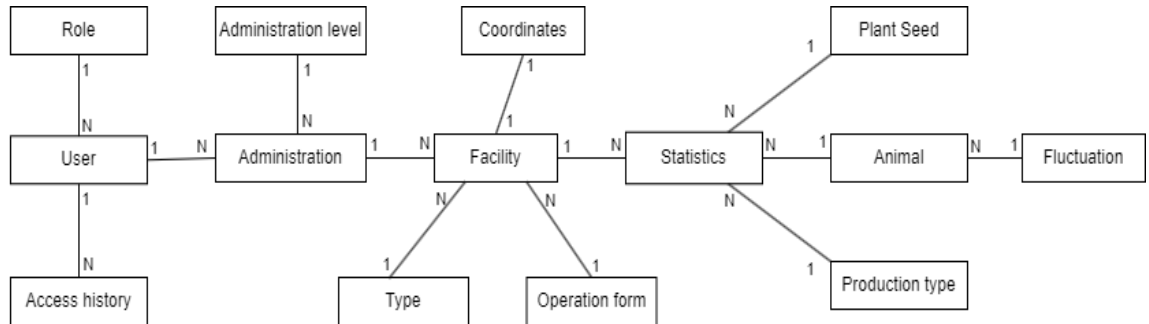


Hình 2.17: Sơ đồ quản lý lưu trữ động vật



Hình 2.18: Sơ đồ quản lý cây giống

2.3 ERD (Entity Relation Diagram)



Hình 2.11 Sơ đồ ERD

2.4 Kết luận

Như vậy, ở chương 2, đã hoàn thành bước phân tích thiết kế, bao gồm vẽ các cơ đồ DFD (mức đỉnh, mức 1, mức dưới đỉnh), DFD và ERD. Từ đó, có một cái nhìn tổng thể, bao quát về những gì phải làm ở bước tiếp theo – bước triển khai.

CHƯƠNG 3: TRIỂN KHAI

Giới thiệu:

Sau khi đã phân tích, thiết kế hệ thống ở chương 2, sẽ đến bước triển khai ở chương 3. Ở chương này, các công việc bao gồm thiết kế cơ sở dữ liệu (lập các bảng thực thể và thuộc tính, từ điển dữ liệu pha phân tích, bảng database, pha thiết kế); viết code và trình bày giao diện web

3.1 Thiết kế cơ sở dữ liệu

3.1.1. Các thực thể và thuộc tính

Bảng 3.1 Các thực thể và thuộc tính

Thực thể (Entity)	Các thuộc tính (Properties)
AdministrativeLevels	<u>id</u> , name
Administrations	<u>code</u> , name , <u>surbordinate</u> , <u>administrativeLevel</u>
Roles	<u>id</u> , name
Users	<u>username</u> , password , firstName , lastName , avatar , email , birthDate , address , isActive , <u>role</u> , otp , otpGeneratedTime , <u>administration</u>
FacilityTypes	<u>id</u> , name
Facilities	<u>code</u> , name , establishmentDate , capacity , isActive, <u>facilitiesType</u> , <u>adminstration</u> , operationType
OperationTypes	<u>name</u> , characteristics
Coordinates	lat, lng , <u>facilities</u>
Fluctuation	<u>id</u> , name , characteristics
AnimalSpecies	<u>name</u> , animalType , image , mainFood , mainDisease , <u>fluctuation</u>
ProductionType	<u>id</u> , woodType , image , capacity
PlantSeed	<u>name</u> , type , image , soilType , mainPest , harvestingPeriod , plantSeason
Statistics	<u>id</u> , unit , quantity , date , <u>aggregates</u>
Aggregates	<u>rowId</u> , isActice , <u>facilities</u> , <u>plantSeed</u> , <u>animalSpecies</u> , <u>productionType</u>
AccessHistory	<u>id</u> , type , decription , <u>username</u>

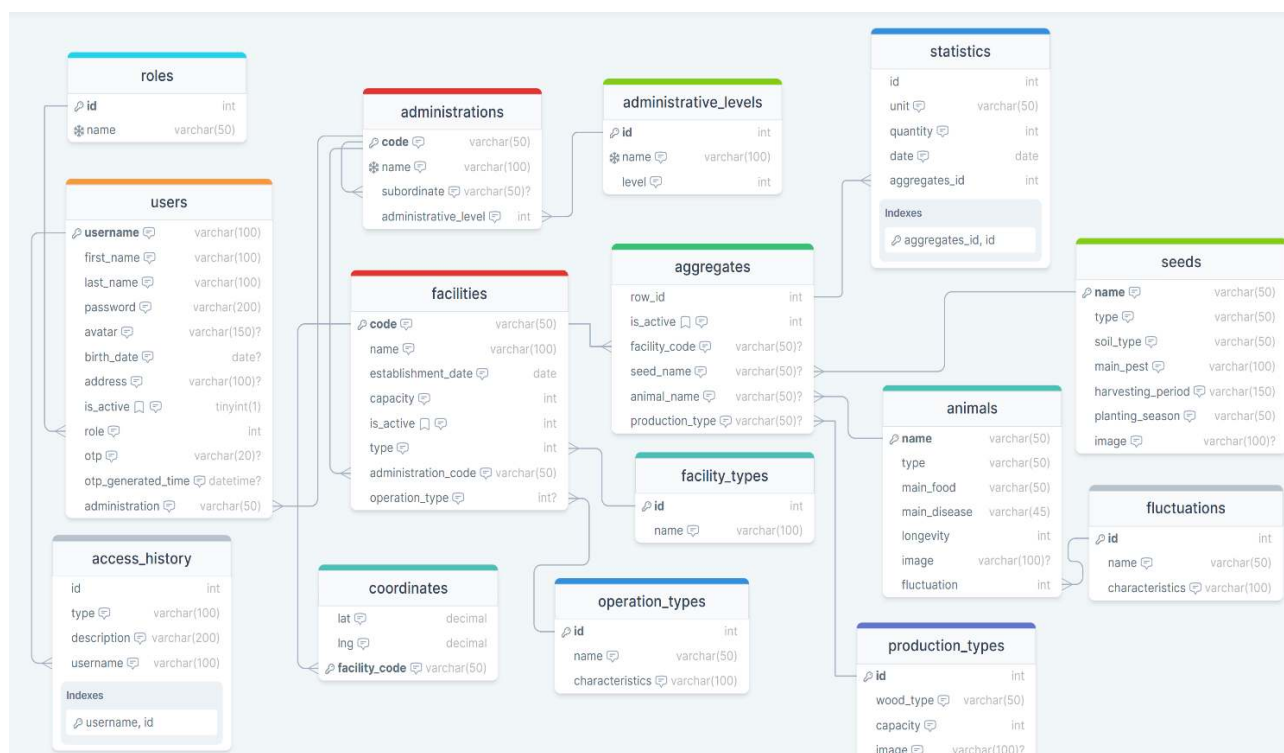
3.1.2. Từ điển dữ liệu pha phân tích:

Bảng 3.2 Bảng phân tích

Tên dữ liệu	Bí danh	Mô tả
Cấp độ hành chính	AdministrativeLevels	Lưu trữ tên và các cấp của cấp độ hành chính
Hành chính	Administrations	Lưu trữ tên, cơ sở trực thuộc, cấp độ hành chính của cơ sở hành chính
Vai trò	Roles	Lưu trữ vai trò của người dùng
Người dùng	Users	Lưu trữ các thông tin cơ bản , tên tài khoản , mật khẩu , email , vai trò và cơ sở hành chính của người dùng
Loại cơ sở	FacilityTypes	Lưu trữ tên của loại cơ sở
Cơ sở	Facilities	Lưu trữ các thông tin cơ bản, loại cơ sở ,sức chứa , cơ sở hành chính của cơ sở
Hình thức hoạt động	OperationTypes	Lưu trữ tên và đặc trưng của hình thức hoạt động
Tọa độ	Coordinates	Lưu trữ kinh độ, vĩ độ của tọa độ và cơ sở có tọa độ đó
Loại biến động	Fluctuation	Lưu trữ tên và đặc trưng của loại biến động
Loài động vật	AnimalSpecies	Lưu trữ các thông tin cơ bản, ảnh, loại biến động của loài động vật
Loại hình sản xuất	ProductionType	Lưu trữ các thông tin cơ bản và ảnh của loại hình sản xuất
Giống cây trồng	PlantSeed	Lưu trữ các thông tin cơ bản và ảnh của giống cây trồng

Số liệu thống kê	Statistics	Lưu trữ đơn vị, số lượng và ngày của số liệu thống kê
Nhóm thống kê	Aggregates	Mô tả quan hệ giữa cơ sở với giống cây trồng, loài động vật và loại hình sản xuất
Lịch sử truy cập	AccessHistory	Lưu trữ lịch sử truy cập của người dùng

3.1.3. Bảng database



Hình 3. 1 Bảng thiết kế database

3.1.4. Pha thiết kế

**Bảng 3.3 Các thuộc tính bảng *administrative_levels*
(CĐHC – cấp độ hành chính)**

Thuộc tính	Kiểu dữ liệu	Chiều dài	Giá trị mẫu	Mô tả	Ghi chú
id	INT		1	Mã CĐHC	NOT NULL
name	VARCHAR	100	Tỉnh	Tên CĐHC	NOT NULL

Bảng 3.4 Các thuộc tính bảng *administrations* (HC – Hành chính)

Thuộc tính	Kiểu dữ liệu	Chiều dài	Giá trị mẫu	Mô tả	Ghi chú
code	VARCHAR	50	HC2	Mã HC	NOT NULL
name	VARCHAR	100	Hà Nam	Tên HC	NOT NULL
subordinate	VARCHAR	50	HC1	Trực thuộc	NOT NULL
administrative_level	INT		1	Cấp độ HC	NOT NULL

Bảng 3.5 Các thuộc tính bảng *roles* (VT – Vai trò)

Thuộc tính	Kiểu dữ liệu	Chiều dài	Giá trị mẫu	Mô tả	Ghi chú
id	INT		1	Mã VT	NOT NULL
name	VARCHAR	100	admin	Tên VT	NOT NULL

Bảng 3.6 Các thuộc tính bảng users (ND – người dùng)

Thuộc tính	Kiểu dữ liệu	Chiều dài	Giá trị mẫu	Mô tả	Ghi chú
username	VARCHAR	100	admin	Tên đăng nhập	NOT NULL
first_name	VARCHAR	100	Nguyễn	Họ	NOT NULL
last_name	VARCHAR	100	Văn A	Tên	NOT NULL
password	VARCHAR	200	Abc123	Mật khẩu	NOT NULL
avatar	VARCHAR	150	(link_avatar)	Avatar	NULL
birth_date	DATE		2023:01:15	Ngày sinh	NULL
address	VARCHAR	100	Hà Nam	Địa chỉ	NULL
is_active	TINYINT	1	1	Trạng thái hđ	NOT NULL
role	INT		1	Vai trò của ND	NOT NULL
otp	VARCHAR	20	123456	Mã OTP	NULL
otp_generated_time	DATE		2023:01:30	Thời gian tạo mã OTP	NULL
administration_code	VARCHAR	50	HC1	Mã hành chính	NOT NULL

Bảng 3.7 Các thuộc tính bảng facility_types (LCS – loại cơ sở)

Thuộc tính	Kiểu dữ liệu	Chiều dài	Giá trị mẫu	Mô tả	Ghi chú
id	INT		1	Mã LCS	NOT NULL
name	VARCHAR	100	Lưu trữ động vật	Tên LCS	NOT NULL

Bảng 3.8 Các thuộc tính bảng facilities (CS – cơ sở)

Thuộc tính	Kiểu dữ liệu	Chiều dài	Giá trị mẫu	Mô tả	Ghi chú
code	VARCHAR	50	CS1	Mã CS	NOT NULL
name	VARCHAR	100	Cơ sở 1	Tên CS	NOT NULL
establishment_date	DATE		2003:01:30	Ngày thành lập	NOT NULL
capacity	INT		10000	Sức chứa	NOT NULL
is_active	TINYINT	1	1	Trạng thái	NOT NULL
type	INT		1	Loại cơ sở	NOT NULL
administration_code	VARCHAR	50	HC1	Mã hành chính	NOT NULL
operation_type	INT		1	Hình thức hđ	NULL

Bảng 3.9 Các thuộc tính bảng operation_types (HTHĐ – hình thức hoạt động)

Thuộc tính	Kiểu dữ liệu	Chiều dài	Giá trị mẫu	Mô tả	Ghi chú
id	INT		1	Mã HTHĐ	NOT NULL
name	VARCHAR	50	Dân dụng	Tên HTHĐ	NOT NULL
characteristics	VARCHAR	100	Dân dụng	Đặc trưng	NOT NULL

Bảng 3.10 Các thuộc tính bảng coordinates (TĐ – tọa độ)

Thuộc tính	Kiểu dữ liệu	Chiều dài	Giá trị mẫu	Mô tả	Ghi chú
lat	DEMICAL		1874.667	Vĩ độ	NOT NULL
lng	DEMICAL		869.352	Kinh độ	NOT NULL
facility_code	VARCHAR	50	CS1	Mã cơ sở của tọa độ	NOT NULL

Bảng 3.11 Các thuộc tính bảng fluctuation (LBĐ – loại biến động)

Thuộc tính	Kiểu dữ liệu	Chiều dài	Giá trị mẫu	Mô tả	Ghi chú
Id	INT		1	Mã LBĐ	NOT NULL
Name	VARCHAR	50	Theo chu kỳ	Tên LBĐ	NOT NULL
Characteristics	VARCHAR	100	BĐ theo chu kỳ	Đặc trưng	NOT NULL

Bảng 3.12 Các thuộc tính bảng animal_species (LĐV – loài động vật)

Thuộc tính	Kiểu dữ liệu	Chiều dài	Giá trị mẫu	Mô tả	Ghi chú
name	VARCHAR	50	Hổ	Tên LĐV	NOT NULL
type	VARCHAR	50	ĐV ăn thịt	Kiểu LĐV	NOT NULL
main_food	VARCHAR	50	Thịt	Thức ăn chính	NOT NULL
main_disease	VARCHAR	45	Sốt	Bệnh chính	NOT NULL
image	VARCHAR	100	(link_image)	Ảnh	NULL
fluctuation	INT		1	Loại biến động	NOT NULL

Bảng 3.13 Các thuộc tính bảng *production_type* (LHSX – loại hình sản xuất)

Thuộc tính	Kiểu dữ liệu	Chiều dài	Giá trị mẫu	Mô tả	Ghi chú
id	INT		1	Mã LHSX	NOT NULL
wood_type	VARCHAR	50	Gỗ lim	Loại gỗ	NOT NULL
capacity	INT		1000	Khả năng sản xuất	NOT NULL
image	VARCHAR	100	(link image)	Ảnh	NULL

Bảng 3.14 Các thuộc tính bảng *plant_seed* (GCT – giống cây trồng)

Thuộc tính	Kiểu dữ liệu	Chiều dài	Giá trị mẫu	Mô tả	Ghi chú
name	VARCHAR	50	Nhãn	Tên GCT	NOT NULL
type	VARCHAR	50	Cây ăn quả	Kiểu GCT	NOT NULL
soil_type	VARCHAR	59	Đất màu mỡ	Loại đất	NOT NULL
main_pest	VARCHAR	100	Bọ xít	Sâu bệnh chính	NOT NULL
harvesting_period	VARCHAR	150	1 tháng	Thời gian thu hoạch	NOT NULL
planting_season	VARCHAR	50	Mùa xuân	Mùa vụ	NOT NULL
image	VARCHAR	100	(link_image)	Ảnh	NULL

Bảng 3.15 Các thuộc tính bảng statistics (SLTK – số liệu thống kê)

Thuộc tính	Kiểu dữ liệu	Chiều dài	Giá trị mẫu	Mô tả	Ghi chú
id	INT		1	Mã SLTK	NOT NULL
unit	VARCHAR	50	Cây	Đơn vị	NOT NULL
quantity	INT		50	Số lượng	NOT NULL
date	DATE		2013:05:15	Ngày thống kê	NOT NULL
aggregates_id	INT		1	Nhóm thống kê	NOT NULL

Bảng 3.16 Các thuộc tính bảng statistics (SLTK – số liệu thống kê)

Thuộc tính	Kiểu dữ liệu	Chiều dài	Giá trị mẫu	Mô tả	Ghi chú
row_id	INT		1	Mã NTK	NOT NULL
is_active	TINYINT	1	0	Trạng thái hđ	NOT NULL
facility_code	VARCHAR	50	CS1	Mã cơ sở	NOT NULL
seed_name	VARCHAR	50	Nhãn	Tên giống cây trồng	NOT NULL
animal_name	VARCHAR	50	Hồ	Tên loài động vật	NOT NULL
production_type	VARCHAR	50	Gỗ lim	Loại hình sản xuất	NOT NULL

Bảng 3.17 Các thuộc tính access_history (LSTC – lịch sử truy cập)

Thuộc tính	Kiểu dữ liệu	Chiều dài	Giá trị mẫu	Mô tả	Ghi chú
id	INT		1	Mã LSTC	NOT NULL
type	VARCHAR	100		Kiểu LSTC	NOT NULL
description	VARCHAR	200		Chi tiết	NOT NULL
username	VARCHAR	100	admin	Tên tài khoản	NOT NULL

3.2 Phần code

3.2.1 Đơn vị hành chính - Administration

Truy xuất toàn bộ đơn vị hành chính

```
public List<Administration> retrieveAllAdministrations() {
    return administrationRepository.findAll();
}
```

Truy xuất đơn vị hành chính theo mã

```
public Administration retrieveAdministrationByCode(String code) {
    Administration administration = this.administrationRepository.
findById(code).orElseThrow(() -> new DataNotFoundException("Not found
administration with code = " + code));
    return administration;
}
```

Truy xuất đơn vị hành chính theo tên

```
public Administration retrieveAdministrationByName(String name) {
    Administration administration =
this.administrationRepository.findById(name)
        .orElseThrow(() -> new DataNotFoundException("Not found
administration with name: " + name));
    return administration;
}
```

Truy xuất toàn bộ đơn vị hành chính trực thuộc có mã là tham số truyền vào

```
public List<AdministrationHierarchyResponse>
retrieveAllSubAdministrations(String code) {
    List<AdministrationHierarchyResponse> res = new ArrayList<>();
    Administration tmp = administrationRepository.findById(code)
        .orElseThrow(() -> new DataNotFoundException("Not found
administration with code = " + code));
    res.add(new AdministrationHierarchyResponse(tmp));
    findChildren(res);
    return res;
}
```

Cập nhật đơn vị hành chính

```
public Administration updateAdministration(String code, AdministrationDTO
administrationDTO) {
    Administration administration = this.administrationRepository
        .findById(code)
        .orElseThrow(() -> new DataNotFoundException(
            "Không tìm thấy đơn vị hành chính với mã : " + code));

    // Kiểm tra cấp hành chính
    String administrativeLevelName =
administrationDTO.getAdministrativeLevelName();
    if
(!administration.getAdministrativeLevel().getName().equals(administrativeLevelName)) {
        AdministrativeLevel administrativeLevel =
this.administrativeLevelRepository
            .findByName(administrationDTO.getAdministrativeLevelName())
            .orElseThrow(() -> new DataNotFoundException(
                "Không thể tìm thấy cấp hành chính: "
                +
administrationDTO.getAdministrativeLevelName()));
        if (administration.getAdministrativeLevel().getLevel() == 1
            && administration.getAdministrativeLevel().getLevel() !=
administrativeLevel.getLevel()) {
            throw new InvalidDataException("Đơn vị hành chính cấp Tỉnh
không thể đổi cấp");
        }
        if (administration.getAdministrativeLevel().getLevel() == 2
            && administration.getAdministrativeLevel().getLevel() !=
administrativeLevel.getLevel()) {
```

```

        throw new InvalidDataException(
            "Đơn vị hành chính cấp Thành phố, Huyện chỉ có thể đổi
cho nhau");
    }
    if(administration.getAdministrativeLevel().getLevel() == 3
        && administration.getAdministrativeLevel().getLevel() !=
administrativeLevel.getLevel()) {
        throw new InvalidDataException(
            "Đơn vị hành chính cấp Phường, Thị trấn, Xã chỉ có thể
đổi cho nhau");
    }
    administration.setAdministrativeLevel(administrativeLevel);
}

// Kiểm tra trực thuộc
String subordinate = administrationDTO.getSubordinateCode() == null ?
"" : administrationDTO.getSubordinateCode();
if (administration.getSubordinate() != null &&
!administration.getSubordinate().getCode().equals(subordinate)) {
    if (administration.getAdministrativeLevel().getLevel() == 1) {
        throw new InvalidDataException(
            "Đơn vị hành chính cấp Tỉnh không thể trực thuộc đơn vị
hành chính khác");
    }
    Administration subordinateAdministration =
this.administrationRepository
        .findById(subordinate)
        .orElseThrow(() -> new DataNotFoundException(
            "Không tìm thấy đơn vị hành chính với mã : " +
subordinate));
    if (administration.getAdministrativeLevel().getLevel() == 2) {
        if
(subordinateAdministration.getAdministrativeLevel().getLevel() != 1){
            throw new InvalidDataException(
                "Đơn vị hành chính cấp Thành phố, Huyện chỉ có thể
trực thuộc đơn vị hành chính cấp tỉnh");
        }
        administration.setSubordinate(subordinateAdministration);
    }
    if (administration.getAdministrativeLevel().getLevel() == 3) {
        if
(subordinateAdministration.getAdministrativeLevel().getLevel() != 3) {
            throw new InvalidDataException(
                "Đơn vị hành chính cấp Phường, Thị trấn, Xã chỉ có
thể trực thuộc đơn vị hành chính cấp thành phố, huyện");
        }
    }
}

```

```

        }
        administration.setSubordinate(subordinateAdministration);
    }
}
administration.setName(administrationDTO.getName());
administration = administrationRepository.save(administration);
return administration;
}

```

Xóa đơn vị hành chính

```

public void deleteByCode(String code) {
    this.administrationRepository.deleteByCode(code);
}

```

3.2.2 Người dùng – User

Lưu file ảnh và trả đường dẫn đến ảnh

```

public String saveImage(MultipartFile avatarFile) {
    if (avatarFile == null) {
        return "";
    }

    // Kiểm tra kích thước file
    if (avatarFile.getSize() > 10 * 1024 * 1024) { // kích thước file phải
<= 10 MB
        throw new InvalidDataException("Kích thước ảnh đại diện phải nhỏ
hơn 10MB");
    }

    // Kiểm tra định dạng file
    String contentType = avatarFile.getContentType();

    if (contentType == null || !contentType.startsWith("image/")) { // Phải
là file ảnh
        throw new InvalidDataException("Ảnh đại diện phải là ảnh");
    }

    // Trích xuất và làm sạch tên file gốc từ hệ thống file của client
    String fileName =
StringUtils.cleanPath(avatarFile.getOriginalFilename());
}

```

```

// Tạo ra một tên file duy nhất
String uniqueFileName = UUID.randomUUID().toString() + "_" + fileName;

// Tạo đường dẫn để lưu file
Path uploadDir = Path.of("uploads");

try {
    if (!Files.exists(uploadDir)) {
        Files.createDirectories(uploadDir);
    }
    Path uploadPath = Path.of(uploadDir.toString(), uniqueFileName);
    Files.copy(avatarFile.getInputStream(), uploadPath,
StandardCopyOption.REPLACE_EXISTING);
} catch (IOException e) {
    throw new RuntimeException(e.getMessage());
}

return uniqueFileName;
}

```

Đăng nhập

```

public String login(LoginDTO loginDTO) {
    User user =
this.userRepository.findByUsernameOrEmail(loginDTO.getUsername())
        .orElseThrow(() -> new DataNotFoundException("Username hoặc mật
khẩu không chính xác"));
    if (user.isActive() != true ||
!user.getPassword().equals(loginDTO.getPassword())) {
        throw new InvalidDataException("Username hoặc mật khẩu không chính
xác");
    }
    return loginDTO.getUsername();
}

```

Cài lại mật khẩu

```

public void resetPassword(ResetPasswordDTO resetPasswordDTO) {
    User user =
this.userRepository.findByEmail(resetPasswordDTO.getEmail())
        .orElseThrow(() -> new DataNotFoundException("Email chưa được
sử dụng"));
    String otp = otpUtil.generateOtp();
}

```

```

        user.setOtp(otp);
        user.setOtpGeneratedTime(LocalDateTime.now());
        this.userRepository.save(user);
        this.emailUtil.sendOtpEmail(user.getUsername(),
resetPasswordDTO.getEmail(), otp);
    }

```

Xác minh mã OTP

```

public String verifyOtp(verifyOtpDTO verifyOtpDTO) {
    User user = this.userRepository.findByEmail(verifyOtpDTO.getEmail())
        .orElseThrow(() -> new DataNotFoundException("Mã xác thực không
chính xác"));
    String otp = user.getOtp();
    if (otp.equals(verifyOtpDTO.getOtp())
        && Duration.between(LocalDateTime.now(),
user.getOtpGeneratedTime()).toSeconds() < (15 * 60)) {
        return verifyOtpDTO.getOtp();
    }
    throw new InvalidDataException("Mã xác thực không chính xác");
}

```

3.2.3 Cơ sở lưu trữ động vật – AnimalStorageFacilities

Thêm một cơ sở lưu trữ động vật

```

public AnimalStorageFacilities
addAnimalStorageFacilities(AnimalStorageFacilitiesDTO
animalStorageFacilitiesDTO,
    String code) {
    AnimalStorageFacilities animalStorageFacilities = new
AnimalStorageFacilities();
    // kiểm tra code
    Optional<AnimalStorageFacilities> tmp1 =
this.animalStorageFacilitiesRepository.findById(code);
    if (tmp1.isPresent()) {
        throw new DataAlreadyExistsException("Đã có cơ sở lưu trữ động vật
với code = " + code);
    }
    // kiểm tra tên

```

```

        Optional<AnimalStorageFacilities> tmp2 =
this.animalStorageFacilitiesRepository
        .findByName(animalStorageFacilitiesDTO.getName());
        if (tmp2.isPresent()) {
            throw new DataAlreadyExistsException(
                "Đã có cơ sở lưu trữ động vật với tên = " +
animalStorageFacilitiesDTO.getName());
        }
        // kiểm tra đơn vị hành chính
        try {
            Administration administration = this.administrationService
                .retrieveAdministrationByCode(animalStorageFacilitiesDTO.ge
tAdministrationCode());
            animalStorageFacilities.setAdministration(administration);
        } catch (Exception exception) {
            throw new DataNotFoundException(
                "Không tìm thấy cơ sở hành chính với mã = " +
animalStorageFacilitiesDTO.getAdministrationCode());
        }
        animalStorageFacilities.setCode(code);
        animalStorageFacilities.setName(animalStorageFacilitiesDTO.getName());
        animalStorageFacilities.setCapacity(animalStorageFacilitiesDTO.getCapac
ity());
        animalStorageFacilities.setDate(animalStorageFacilitiesDTO.getDate());
        animalStorageFacilities.setDetail(animalStorageFacilitiesDTO.getDetail(
));

        return animalStorageFacilitiesRepository.save(animalStorageFacilities);
    }

```

Xóa 1 cơ sở lưu trữ động vật theo mã

```

public void deleteAnimalStorageFacilitiesByCode(String code) {
    AnimalStorageFacilities animalStorageFacilities =
this.getAnimalStorageFacilitiesByCode(code);
    this.animalStorageFacilitiesRepository.deleteById(animalStorageFaciliti
es.getCode());
}

```

3.2.4 Tọa độ trên bản đồ - Coordinates DTO

Cập nhật tọa độ

```
public CoordinatesDTO updateCoordinates(CoordinatesDTO coordinatesDTO) {
    AnimalStorageFacilities animalStorageFacilities = this
        .getAnimalStorageFacilitiesByCode(coordinatesDTO.getCode());
    animalStorageFacilities.setLat(coordinatesDTO.getLat());
    animalStorageFacilities.setLng(coordinatesDTO.getLng());
    this.animalStorageFacilitiesRepository.save(animalStorageFacilities);
    return coordinatesDTO;
}
```

Xóa tọa độ

```
public void deleteCoordinates(String code) {
    AnimalStorageFacilities animalStorageFacilities = this
        .getAnimalStorageFacilitiesByCode(code);
    animalStorageFacilities.setLat("");
    animalStorageFacilities.setLng("");
    this.animalStorageFacilitiesRepository.save(animalStorageFacilities);
}
```

3.2.5 Loài động vật – AnimalSpecies

Thêm 1 loài động vật

```
public AnimalSpecies addAnimalSpecies(AnimalSpeciesDTO animalSpeciesDTO,
    MultipartFile imageFile) {

    // kiểm tra tên
    Optional<AnimalSpecies> tmp =
    this.animalSpeciesRepository.findById(animalSpeciesDTO.getName());
    if (tmp.isPresent()) {
        throw new DataAlreadyExistsException("Đã tồn tại loài động vật với
    tên = " + animalSpeciesDTO.getName());
    }
    String image = this.saveImage(imageFile);
    Fluctuation fluctuation =
    this.getFluctuationById(animalSpeciesDTO.getFluctuationId());

    AnimalSpecies animalSpecies = AnimalSpecies.builder()
```

```

        .name(animalSpeciesDTO.getName())
        .animalType(animalSpeciesDTO.getAnimalType())
        .image(image)
        .mainFood(animalSpeciesDTO.getMainFood())
        .mainDisease(animalSpeciesDTO.getMainDisease())
        .longevity(animalSpeciesDTO.getLongevity())
        .fluctuation(fluctuation)
        .build();

    return animalSpeciesRepository.save(animalSpecies);
}

```

Lấy tất cả loài động vật trong 1 cơ sở

```

public List<AnimalSpecies> getAllAnimalSpeciesByFacilitiesCode(String code) {
    List<AnimalSpecies> listAnimalSpecies =
    this.asfAsRelationshipRepository
        .selectAllAnimalSpeciesByFacilitiesCode(code)
        .orElseThrow(() -> new DataNotFoundException("cơ sở lưu trữ này
không có loài động vật nào"));
    return listAnimalSpecies;
}

```

3.2.6 Loại biến động – Fluctuation

Truy xuất loại biến động theo mã

```

public Fluctuation getFluctuationById(int id) {
    return this.fluctuationRepository.findById(id)
        .orElseThrow(() -> new DataNotFoundException("Không tìm thấy
loại biến động với id = " + id));
}

```

3.2.7 Thống kê theo tháng/quý/năm

Hàm hỗ trợ thống kê

```

public Long getMonthQuantityOfFacilities(String name, int month, int year) {
    YearMonth yearMonth = YearMonth.of(year, month);
    int day = yearMonth.lengthOfMonth();
    LocalDate localDate = LocalDate.of(year, month, day);
    Date date = Date.valueOf(localDate);
}

```

```

        return
this.asfAsRelationshipRepository.getMonthQuantityOfFacilities(name, date)
        .orElseThrow(() -> new DataNotFoundException(
            "Không tồn tại dữ liệu của " + name + " trong tháng " +
month + " năm " + year));
    }

```

```

public long getMonthQuantityOfFacilities(String name, LocalDate date) {
    LocalDate date2 = this.caculateDate(date);
    Date dateExisting = Date.valueOf(date2);
    try {
        long quantity =
this.asfAsRelationshipRepository.getMonthQuantityOfFacilitiesNew(name,
dateExisting);
        return quantity;
    } catch (Exception e) {
        return 0;
    }
}

```

```

public int getTotalBeforeFuntion(String name, LocalDate date, int i) {
    LocalDate date2 = this.caculateDate(date);
    Date dateExisting = Date.valueOf(date2);
    try {
        long quantity =
this.asfAsRelationshipRepository.getMonthQuantityOfFacilitiesNew(name,
dateExisting);
        return i + 1;
    } catch (Exception e) {
        return i;
    }
}

```

Thống kê số lượng theo tháng trong tất cả cơ sở

Hàm hỗ trợ

```

public List<FacilitiesQuantity> getQuantityOfFacilitiesBeforeTime(LocalDate
date) {
    List<FacilitiesQuantity> facilitiesQuantities =
this.asfAsRelationshipRepository
        .selectAllQuantityOfFacilities(Date.valueOf(date));
    if (facilitiesQuantities.isEmpty()) {
        throw new DataNotFoundException(

```

```

        "Dữ liệu trong tháng " + date.getMonthValue() + " năm " +
date.getYear() + " chưa tồn tại");
    }
    return facilitiesQuantities;
}

```

Hàm chính

```

public List<FacilitiesQuantityInMoth> getQuantityOfFacilitiesWithTime(LocalDate
beginDate, LocalDate endDate) {
    List<FacilitiesQuantityInMoth> allData = new ArrayList<>();
    beginDate = this.caculateDate(beginDate);
    endDate = this.caculateDate(endDate);
    if (endDate.isAfter(LocalDate.now())) {
        endDate = LocalDate.now();
        while (beginDate.isBefore(endDate.plusDays(1))) {
            List<FacilitiesQuantity> data =
this.getQuantityOfFacilitiesBeforeTime(beginDate);
            FacilitiesQuantityInMoth tmp =
FacilitiesQuantityInMoth.builder().date(beginDate).data(data).build();
            allData.add(tmp);
            beginDate = beginDate.plusMonths(1);
            beginDate = this.caculateDate(beginDate);
        }
        List<FacilitiesQuantity> data =
this.getQuantityOfFacilitiesBeforeTime(endDate);
        FacilitiesQuantityInMoth tmp =
FacilitiesQuantityInMoth.builder().date(endDate).data(data).build();
        allData.add(tmp);
    } else {
        while (beginDate.isBefore(endDate.plusDays(1))) {
            List<FacilitiesQuantity> data =
this.getQuantityOfFacilitiesBeforeTime(beginDate);
            FacilitiesQuantityInMoth tmp =
FacilitiesQuantityInMoth.builder().date(beginDate).data(data).build();
            allData.add(tmp);
            beginDate = beginDate.plusMonths(1);
            beginDate = this.caculateDate(beginDate);
        }
    }
    return allData;
}

```

Thống kê số lượng theo quý trong tất cả cơ sở

Hàm hỗ trợ

```
public String changeToQuarter(LocalDate date) {
    int month = date.getMonthValue();
    int year = date.getYear();

    return (month >= 1 && month <= 3) ? "Q1-" + year
        : (month >= 4 && month <= 6) ? "Q2-" + year : (month >= 7 &&
month <= 9) ? "Q3-" + year : "Q4-" + year;
}

public LocalDate getLastDayOfQuarter(LocalDate date) {
    LocalDate firstDayOfQuarter =
date.with(date.getMonth().firstMonthOfQuarter())
        .with(TemporalAdjusters.firstDayOfMonth());
    LocalDate lastDayOfQuarter = firstDayOfQuarter.plusMonths(2)
        .with(TemporalAdjusters.lastDayOfMonth());
    return lastDayOfQuarter;
}

public LocalDate getFirstDayOfQuarter(LocalDate date) {
    LocalDate firstDayOfQuarter =
date.with(date.getMonth().firstMonthOfQuarter())
        .with(TemporalAdjusters.firstDayOfMonth());
    return firstDayOfQuarter;
}

public List<FacilitiesQuantity> getQuarterFacilitiesWithStartDate(LocalDate
startDate) {
    LocalDate dateTmp = startDate;
    List<FacilitiesQuantity> list = new ArrayList<>();
    LocalDate lastDayOfQuarter = this.getLastDayOfQuarter(startDate);
    List<String> listASF = this.animalStorageFacilitiesRepository
        .findAllFacilitiesNameBeforeTime(Date.valueOf(lastDayOfQuarter)
);
    for (String asf : listASF) {
        long sum = 0;
        int i = 0;
        for (int j = 1; j <= 3; j++) {
            sum += this.getMonthQuantityOfFacilities(asf, startDate);
            i = this.getTotalBeforeFuntion(asf, startDate, i);

            startDate = startDate.plusMonths(1);
        }
        if (i == 0) {
```

```

        throw new DataNotFoundException("Dữ liệu trong " +
this.changeToQuarter(startDate) + " của "
        + asf + " không tồn tại");
    }
    long quantity = sum / i;
    FacilitiesQuantity facilitiesQuantity = new FacilitiesQuantity(asf,
quantity);

    list.add(facilitiesQuantity);
    startDate = dateTmp;
}
return list;
}

```

Hàm chính

```

public List<FacilitiesQuantityInQuarter>
getQuarterQuantityOfFacilitiesWithTime(LocalDate startDate,
    LocalDate endDate) {
    if
(this.changeToQuarter(startDate).equals(this.changeToQuarter(endDate))
        || startDate.isAfter(endDate)) {
        throw new DataAccessException(
            "Ngày bắt đầu và ngày kết thúc không hợp lệ .Vui lòng nhập
lại");
    }
    List<FacilitiesQuantityInQuarter> list = new ArrayList<>();
    while (true) {
        FacilitiesQuantityInQuarter tmp = new
FacilitiesQuantityInQuarter();
        List<FacilitiesQuantity> data =
this.getQuarterFacilitiesWithStartDate(startDate);
        tmp.setQuarter(this.changeToQuarter(startDate));
        tmp.setData(data);
        list.add(tmp);
        LocalDate firstDayNextQuarter =
this.getLastDayOfQuarter(startDate).plusDays(1);
        startDate = firstDayNextQuarter;
        if (startDate.isAfter(endDate))
            break;
    }
    return list;
}

```

Thống kê số lượng theo năm trong tất cả cơ sở

Hàm hỗ trợ

```
public List<FacilitiesQuantity> getYearQuantityFacilitiesWithYear(int year) {
    List<FacilitiesQuantity> list = new ArrayList<>();
    String endDate = year + "-12-30";
    LocalDate startDate = LocalDate.of(year, 1, 1);
    LocalDate dateTmp = startDate;
    // LocalDate endDate = LocalDate.of(year, 12, 30);
    List<String> listASF = this.animalStorageFacilitiesRepository
        .findAllFacilitiesNameBeforeTime(Date.valueOf(endDate));
    for (String asf : listASF) {
        long sum = 0;
        int i = 0;
        for (int j = 1; j <= 12; j++) {
            sum += this.getMonthQuantityOfFacilities(asf, startDate);
            i = this.getTotalBeforeFuntion(asf, startDate, i);
            startDate = startDate.plusMonths(1);
        }
        if (i == 0) {
            throw new DataNotFoundException(
                "Dữ liệu trong năm " + year + " của " + asf + " không
tồn tại");
        }
        long quantity = sum / i;
        FacilitiesQuantity facilitiesQuantity = new FacilitiesQuantity(asf,
quantity);
        list.add(facilitiesQuantity);
        startDate = dateTmp;
    }
    return list;
}
```

Hàm chính

```
public List<FacilitiesQuantityInYear> getYearQuantityOfFacilitiesWithTime(int
startYear, int endYear) {
    List<FacilitiesQuantityInYear> list = new ArrayList<>();
    if (startYear > endYear) {
        throw new DataAccessResourceFailureException("Năm bắt đầu và năm
kết thúc không hợp lệ .Vui lòng nhập lại");
    }
    if (startYear == endYear) {
        List<FacilitiesQuantity> data =
this.getYearQuantityFacilitiesWithYear(endYear);
```

```

        FacilitiesQuantityInYear tmp = new
FacilitiesQuantityInYear(endYear, data);
        list.add(tmp);
        return list;
    }
    while (true) {
        FacilitiesQuantityInYear tmp = new FacilitiesQuantityInYear();
        List<FacilitiesQuantity> data =
this.getYearQuantityFacilitiesWithYear(startYear);
        tmp.setYear(startYear);
        tmp.setData(data);
        list.add(tmp);
        int nextYear = startYear + 1;
        if (nextYear == endYear) {
            List<FacilitiesQuantity> data2 =
this.getYearQuantityFacilitiesWithYear(endYear);
            FacilitiesQuantityInYear tmp2 = new
FacilitiesQuantityInYear(endYear, data2);
            list.add(tmp2);
            break;
        }
        startYear = nextYear;
    }
    return list;
}

```

3.2.8 Cơ sở sản xuất giống cây trồng - PlantFacilities

Thêm một cơ sở sản xuất giống cây trồng

```

public PlantFacilities addPlantFacilities(PlantFacilitiesDTO
plantFacilitiesDTO, String code) {
    PlantFacilities plantFacilities = new PlantFacilities();

    // kiểm tra code
    Optional<PlantFacilities> tmp1 =
this.plantFacilitiesRepository.findById(code);
    if (tmp1.isPresent()) {
        throw new DataAlreadyExistsException("Đã có cơ sở sản xuất giống
cây trồng với code = " + code);
    }
    // kiểm tra tên
    Optional<PlantFacilities> tmp2 = this.plantFacilitiesRepository

```

```

        .findByName(plantFacilitiesDTO.getName());
    if (tmp2.isPresent()) {
        throw new DataAlreadyExistsException(
            "Đã có cơ sở sản xuất giống cây trồng với tên = " +
plantFacilitiesDTO.getName());
    }
    // kiểm tra đơn vị hành chính
    try {
        Administration administration = this.administrationService
            .retrieveAdministrationByCode(plantFacilitiesDTO.getAdminis-
trationCode());
        plantFacilities.setAdministration(administration);
    } catch (Exception exception) {
        throw new DataNotFoundException(
            "Không tìm thấy cơ sở hành chính với code = " +
plantFacilitiesDTO.getAdministrationCode());
    }

    plantFacilities.setCode(code);
    plantFacilities.setName(plantFacilitiesDTO.getName());
    plantFacilities.setCapacity(plantFacilitiesDTO.getCapacity());
    plantFacilities.setDate(plantFacilitiesDTO.getDate());

    return plantFacilitiesRepository.save(plantFacilities);
}

```

Xóa 1 cơ sở sản xuất giống cây trồng

```

public void deletePlantFacilitiesByCode(String code) {
    PlantFacilities plantFacilities = this.getPlantFacilitiesByCode(code);
    this.plantFacilitiesRepository.deleteById(plantFacilities.getCode());
}

```

3.2.9 Giống cây trồng

Thêm 1 giống cây trồng

```
public PlantSeed addPlantSeed(PlantSeedDTO plantSeedDTO, MultipartFile
imageFile) {
    // kiểm tra tên
    Optional<PlantSeed> tmp =
this.plantSeedRepository.findById(plantSeedDTO.getName());
    if (tmp.isPresent()) {
        throw new DataAlreadyExistsException("Đã tồn tại giống cây trồng
với tên = " + plantSeedDTO.getName());
    }
    String image = this.saveImage(imageFile);

    PlantSeed plantSeed = PlantSeed.builder()
        .name(plantSeedDTO.getName())
        .mainPest(plantSeedDTO.getMainPest())
        .image(image)
        .type(plantSeedDTO.getType())
        .soilType(plantSeedDTO.getSoilType())
        .plantSeason(plantSeedDTO.getPlantSeason())
        .harvestingPeriod(plantSeedDTO.getHarvestingPeriod())
        .build();

    return plantSeedRepository.save(plantSeed);
}
```

Xóa 1 giống cây trồng

```
public void deletePlantSeedByName(String name) {
    PlantSeed plantSeed = this.getPlantSeedByName(name);
    this.plantSeedRepository.deleteById(plantSeed.getName());
}
```

Truy xuất toàn bộ giống cây trồng trong 1 cơ sở

```
public List<PlantSeed> getAllPlantSeedByFacilitiesCode(String code) {
    List<PlantSeed> plantSeeds = this.pfPsRelationshipRepository
        .selectAllPlantSeedByFacilitiesCode(code)
        .orElseThrow(() -> new DataNotFoundException("cơ sở sản xuất
này không có giống cây trồng nào"));
    return plantSeeds;
}
```

3.2.10 Cơ sở sản xuất chế biến gỗ - WoodFacilities

Thêm 1 cơ sở chế biến gỗ

```
public WoodFacilities addWoodFacilities(WoodFacilitiesDTO woodFacilitiesDTO,
String code) {
    WoodFacilities woodFacilities = new WoodFacilities();

    // kiểm tra code
    Optional<WoodFacilities> tmp1 =
this.woodFacilitiesRepository.findById(code);
    if (tmp1.isPresent()) {
        throw new DataAlreadyExistsException("Đã có cơ sở sản xuất gỗ với
code = " + code);
    }
    // kiểm tra tên
    Optional<WoodFacilities> tmp2 = this.woodFacilitiesRepository
        .findByName(woodFacilitiesDTO.getName());
    if (tmp2.isPresent()) {
        throw new DataAlreadyExistsException(
            "Đã có cơ sở sản xuất gỗ với tên = " +
woodFacilitiesDTO.getName());
    }
    // kiểm tra đơn vị hành chính
    try {
        Administration administration = this.administrationService
            .retrieveAdministrationByCode(woodFacilitiesDTO.getAdminstr
ationCode());
        woodFacilities.setAdministration(administration);
    } catch (Exception exception) {
        throw new DataNotFoundException(
            "Không tìm thấy cơ sở hành chính với mã = " +
woodFacilitiesDTO.getAdminstrationCode());
    }
    // kiểm tra hình thức hoạt động
    OperationForm operationForm =
this.getOperationFormByName(woodFacilitiesDTO.getOperationFormName());

    woodFacilities.setCode(code);
    woodFacilities.setName(woodFacilitiesDTO.getName());
    woodFacilities.setCapacity(woodFacilitiesDTO.getCapacity());
    woodFacilities.setDate(woodFacilitiesDTO.getDate());
    woodFacilities.setOperationForm(operationForm);
}
```

```

        return woodFacilitiesRepository.save(woodFacilities);
    }

```

Xóa 1 cơ sở chế biến gỗ

```

public void deleteWoodFacilitiesByCode(String code) {
    WoodFacilities woodFacilities = this.getWoodFacilitiesByCode(code);
    this.woodFacilitiesRepository.deleteById(woodFacilities.getCode());
}

```

3.2.11 Loại hình sản xuất – ProductionType

Thêm 1 loại hình sản xuất

```

public ProductionType addProductionType(ProductionTypeDTO productionTypeDTO,
    MultipartFile imageFile) {
    // kiểm tra tên
    Optional<ProductionType> tmp =
this.productionTypeRepository.findById(productionTypeDTO.getWoodType());
    if (tmp.isPresent()) {
        throw new DataAlreadyExistsException(
            "Đã tồn tại loại hình sản xuất với loại gỗ = " +
productionTypeDTO.getWoodType());
    }
    String image = this.saveImage(imageFile);

    ProductionType productionType = ProductionType.builder()
        .woodType(productionTypeDTO.getWoodType())
        .capacity(productionTypeDTO.getCapacity())
        .image(image)
        .build();

    return productionTypeRepository.save(productionType);
}

```

Xóa 1 loại hình sản xuất

```

public void deleteProductionTypeByWoodName(String name) {
    ProductionType productionType = this.getProductionTypeByWoodName(name);
    this.productionTypeRepository.deleteById(productionType.getWoodType());
}

```

Truy xuất toàn bộ loại hình sản xuất trong 1 cơ sở chế biến gỗ

```
public List<ProductionType> getAllProductionTypeByFacilitiesCode(String code)
{
    List<ProductionType> productionTypes = this.wfPtRelationshipRepository
        .selectAllProductionTypeByFacilitiesCode(code)
        .orElseThrow(() -> new DataNotFoundException("cơ sở sản xuất
này không có loại hình sản xuất nào"));
    return productionTypes;
}
```

3.2.12 Hình thức hoạt động – OperationForm

Thêm một hình thức hoạt động

```
public OperationForm addOperationForm(OperationFormDTO operationFormDTO, String
name) {
    // kiểm tra tên
    Optional<OperationForm> tmp =
this.operationFormRepository.findById(operationFormDTO.getName());
    if (tmp.isPresent()) {
        throw new DataAlreadyExistsException(
            "Đã tồn tại hình thức hoạt động với tên = " +
operationFormDTO.getName());
    }

    OperationForm operationForm = OperationForm.builder()
        .name(operationFormDTO.getName())
        .description(operationFormDTO.getDescription())
        .build();

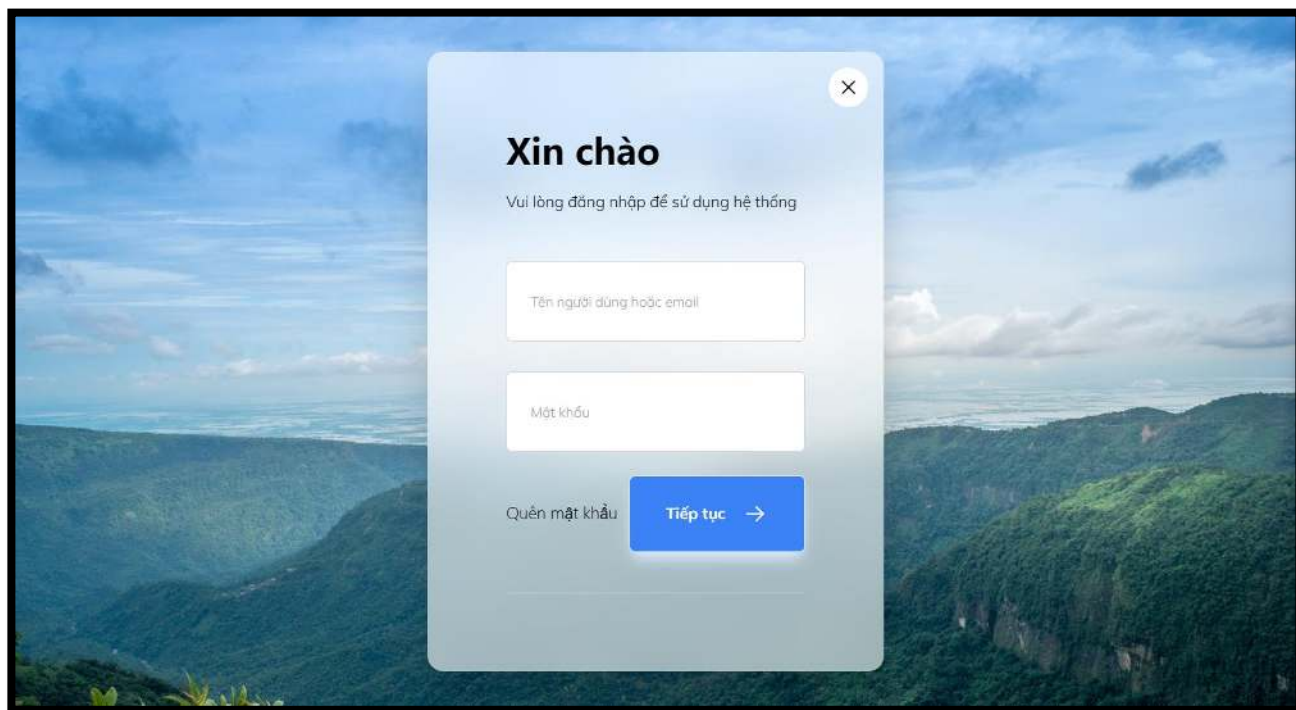
    return operationFormRepository.save(operationForm);
}
```

Xóa 1 hình thức hoạt động

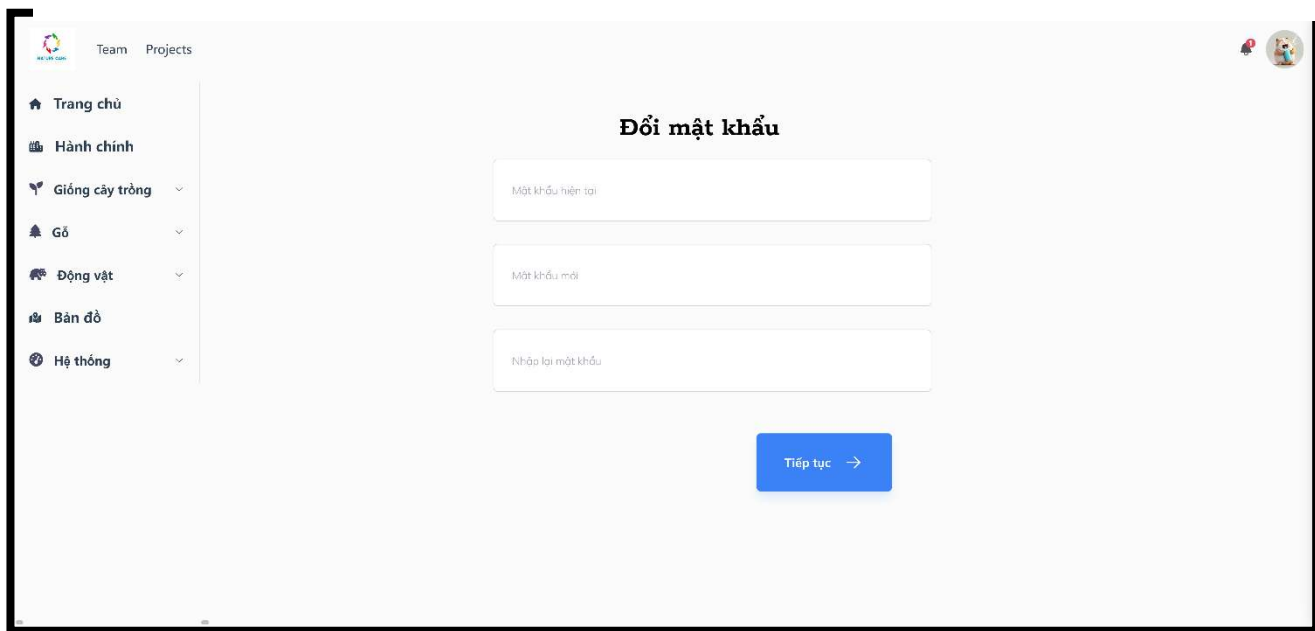
```
public void deleteOperationFormByName(String name) {
    try {
        OperationForm operationForm = this.getOperationFormByName(name);
        this.operationFormRepository.deleteById(operationForm.getName());
    } catch (Exception e) {
        throw new InvalidDataException("Hình thức hoạt động này vẫn đang
được sử dụng trong các cơ sở");
    }
}
```

3.3 Giao diện trực quan

3.3.1 Giao diện đăng nhập



Hình 3. 3- Giao diện đăng nhập

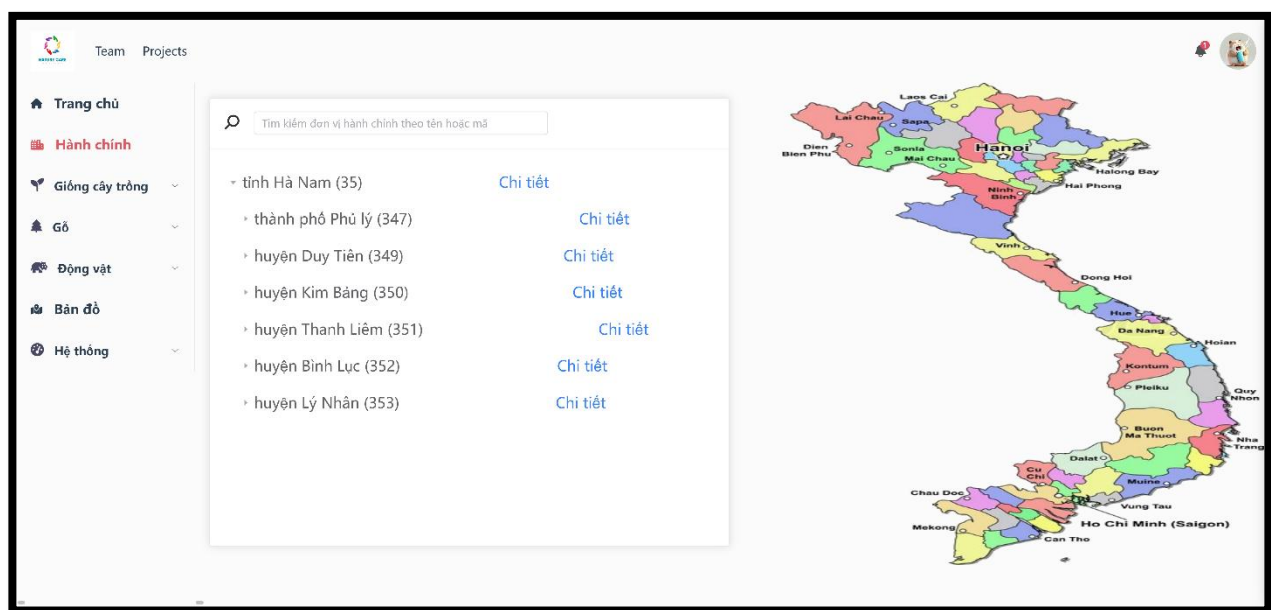


Hình 3. 2- Giao diện đổi mật khẩu

3.3.2 Giao diện chính

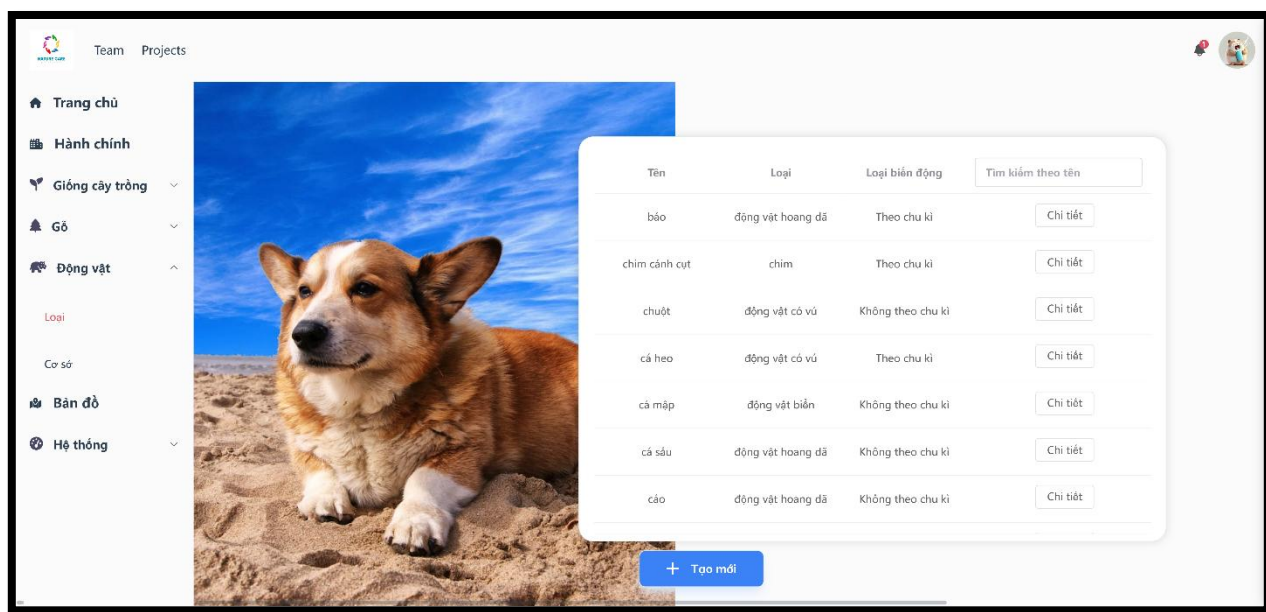


Hình 3. 4. Trang chủ, logo dự án

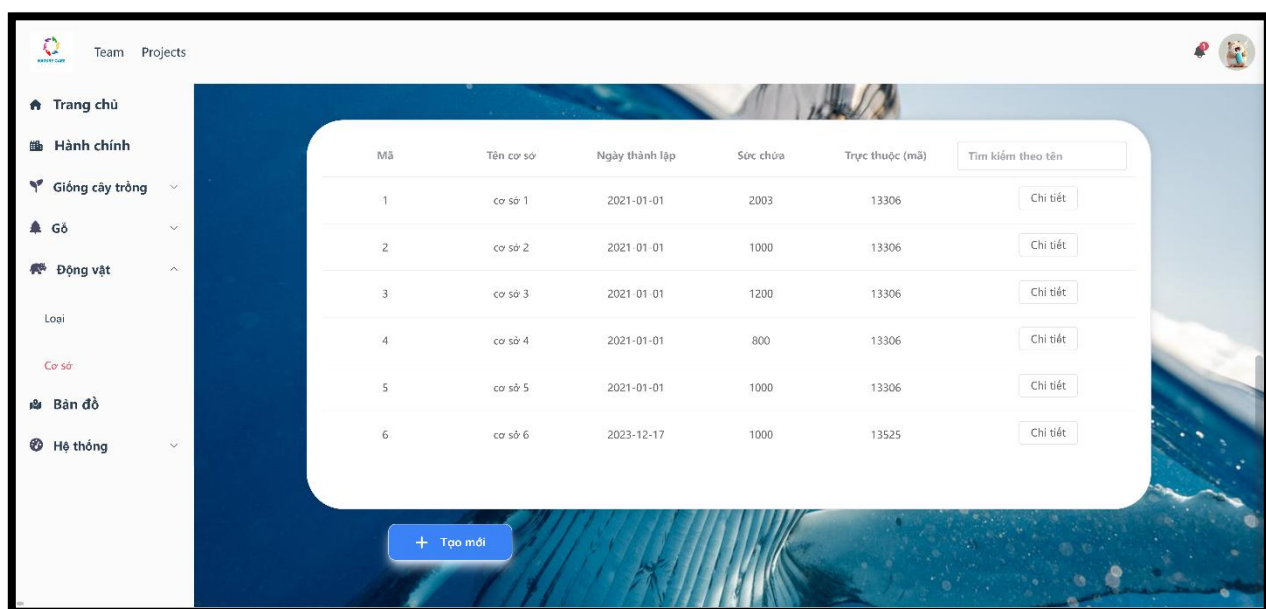


Hình 3. 5. Quản lý hành chính

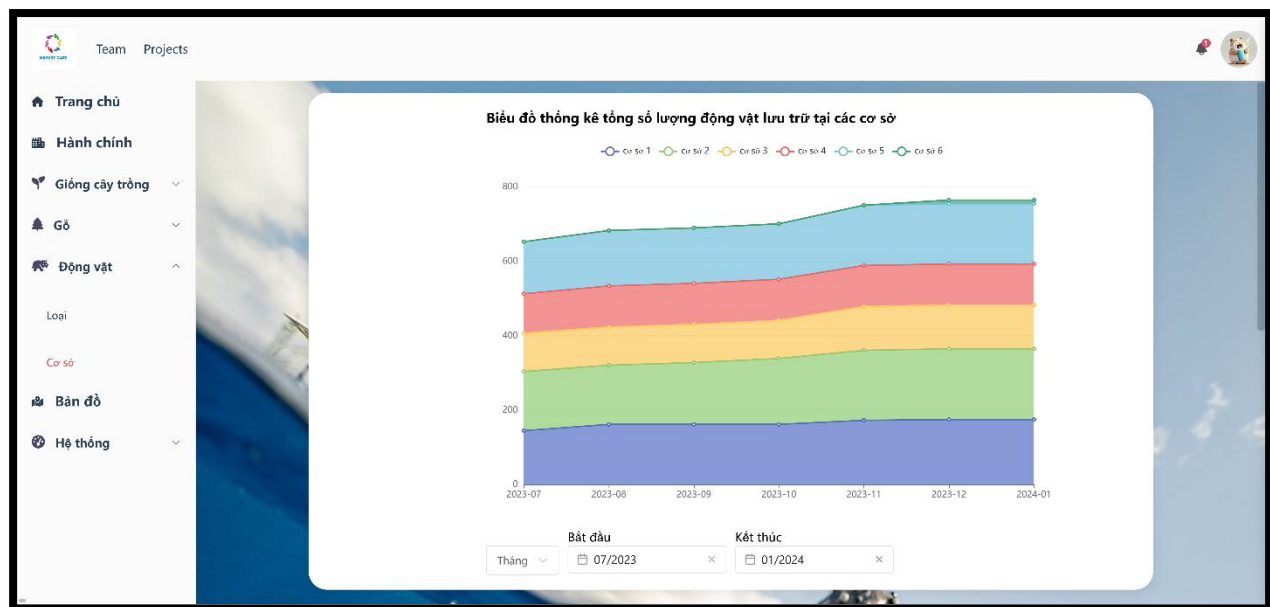
3.3.3 Giao diện quản lý động vật



Hình 3. 6: Giao diện các loại động vật

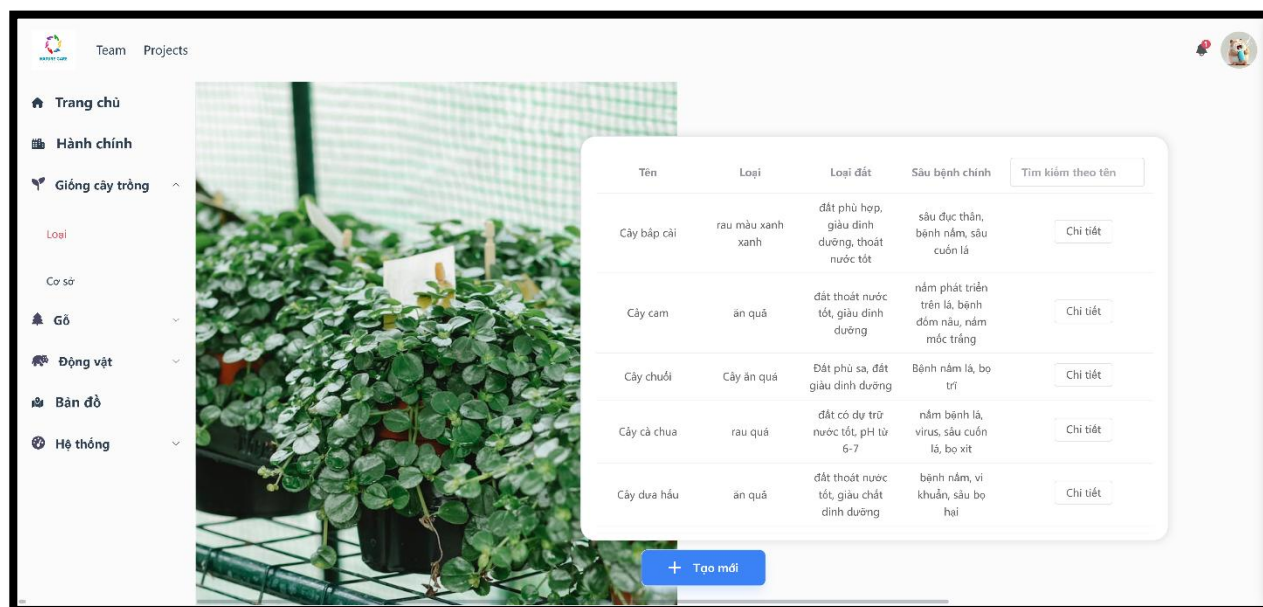


Hình 3.7: Giao diện cơ sở lưu trữ động vật

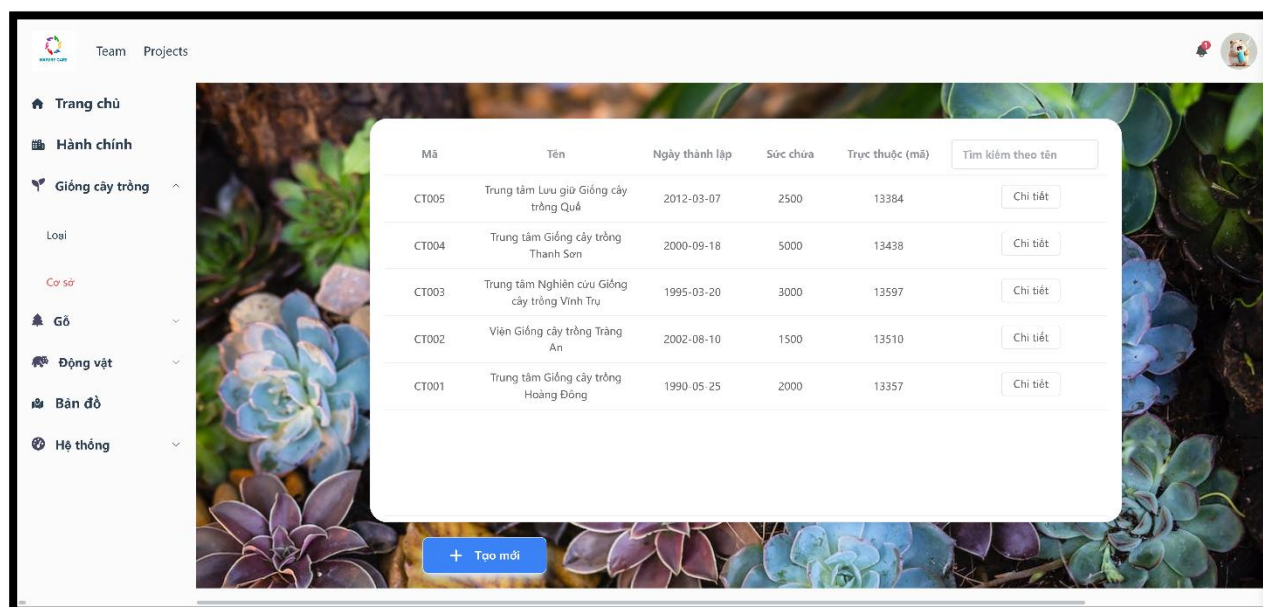


Hình 3.8. Giao diện biểu đồ thống kê tổng số lượng động vật tại các cơ sở

3.3.4 Giao diện quản lý giống cây trồng

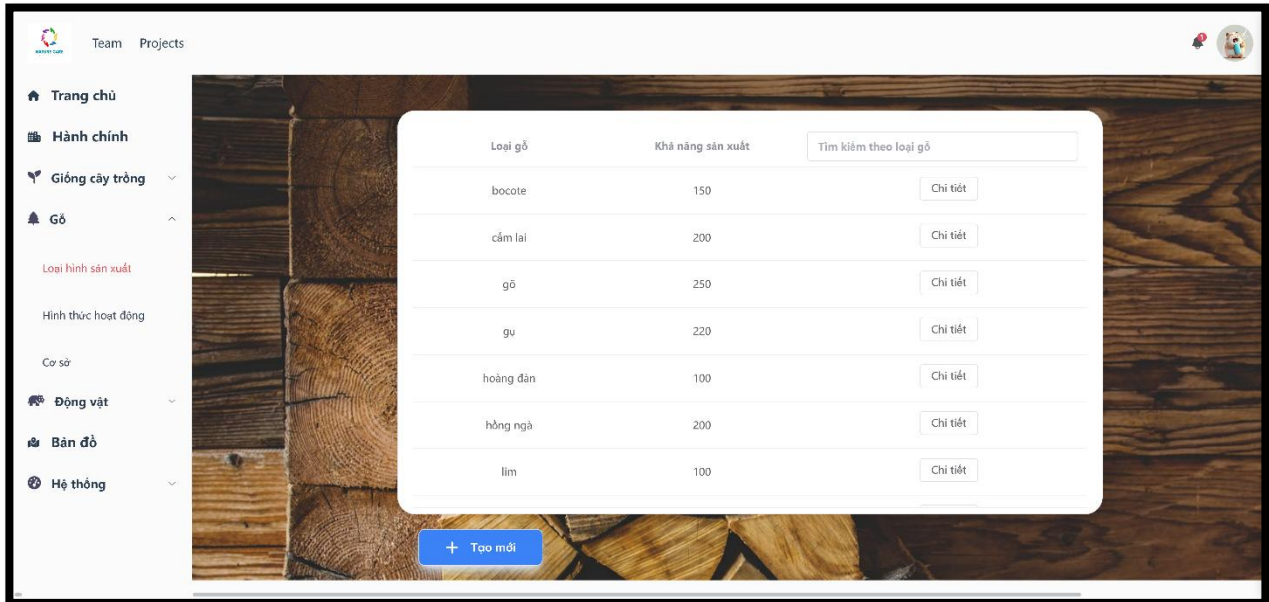


Hình 3.9: Giao diện các Loại cây trồng

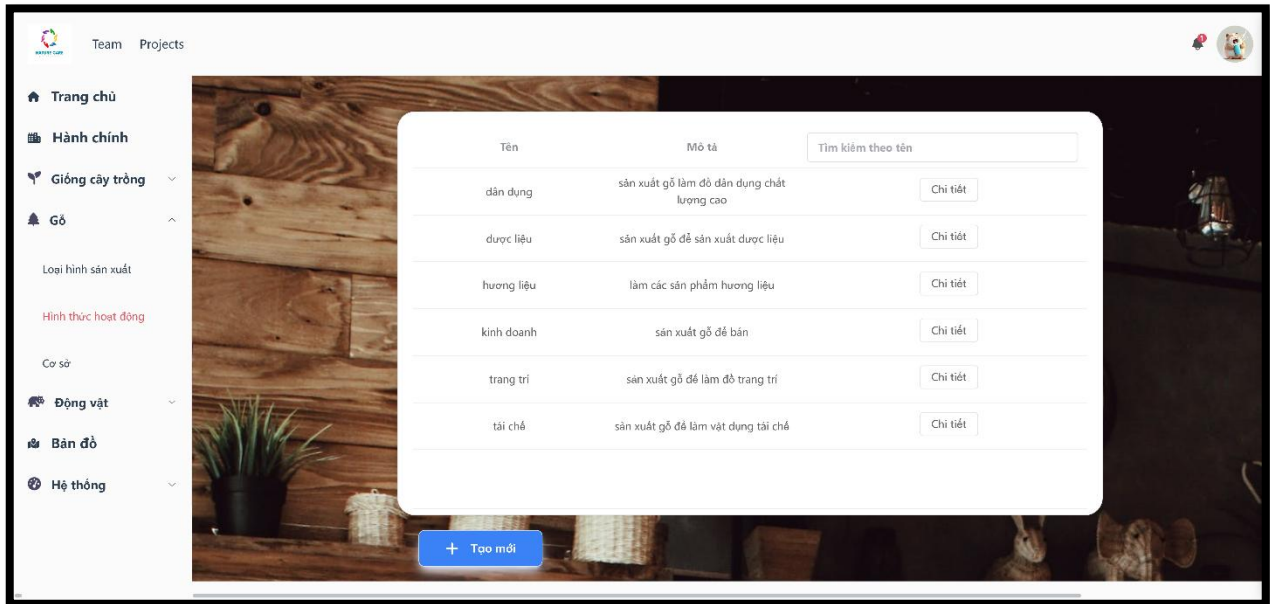


Hình 3.10: Giao diện cơ sở sản xuất giống cây trồng

3.3.5 Giao diện quản lý cơ sở sản xuất gỗ



Hình 3.11: Giao diện các Loại hình sản xuất gỗ

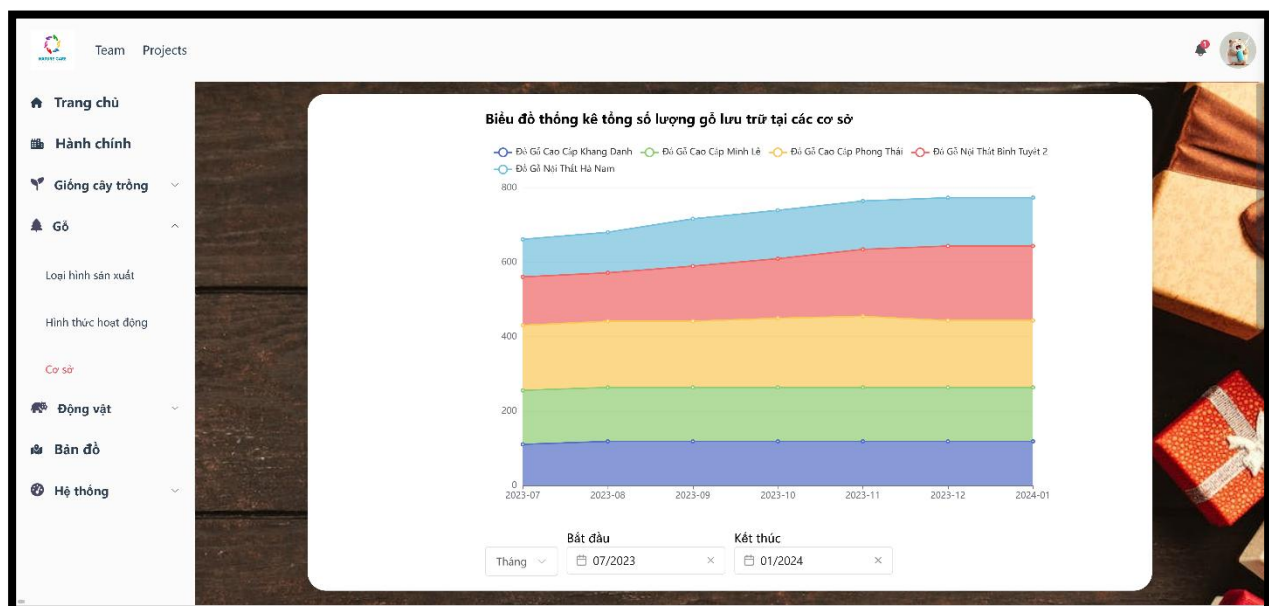


Hình 3.12: Giao diện các Hình thức hoạt động cơ sở sản xuất gỗ

Mã	Tên cơ sở	Ngày thành lập	Sức chứa	Trục thuộc (mã)	Tim kiếm theo tên
1	Đồ Gỗ Cao Cấp Phong Thái	2021-10-30	2000	13450	Chi tiết
2	Đồ Gỗ Cao Cấp Khang Danh	2018-09-02	3000	13321	Chi tiết
3	Đồ Gỗ Cao Cấp Minh Lê	2017-04-25	3000	13324	Chi tiết
4	Đồ Gỗ Nội Thất Hà Nam	2016-03-24	3500	13420	Chi tiết
5	Đồ Gỗ Nội Thất Bình Tuyết 2	2015-04-14	5000	13318	Chi tiết

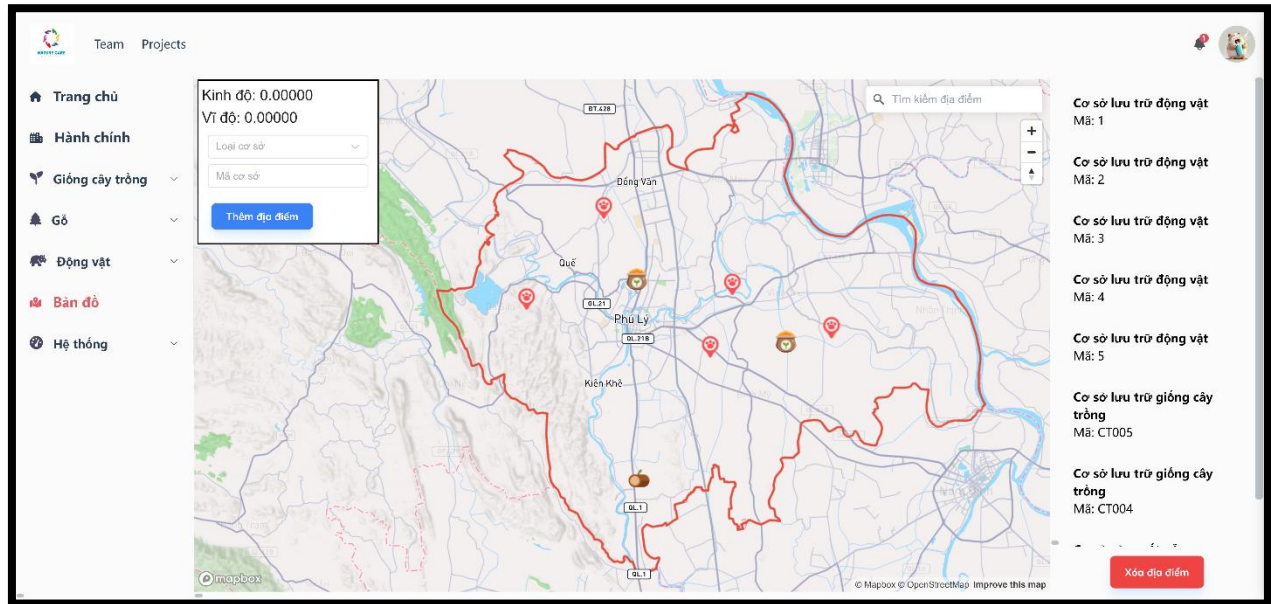
[+ Tạo mới](#)

Hình 3.13. Giao diện các Cơ sở sản xuất gỗ



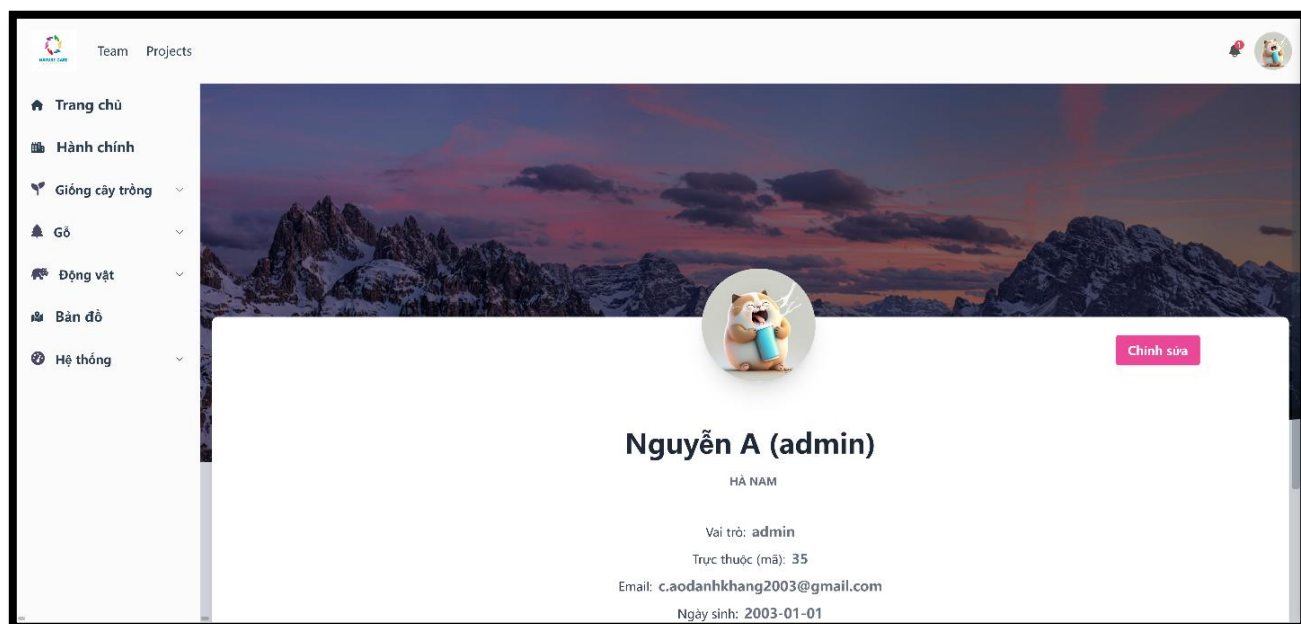
Hình 3.14. Giao diện Biểu đồ tổng số lượng gỗ lưu trữ tại các cơ sở

3.3.6 Giao diện Bản đồ

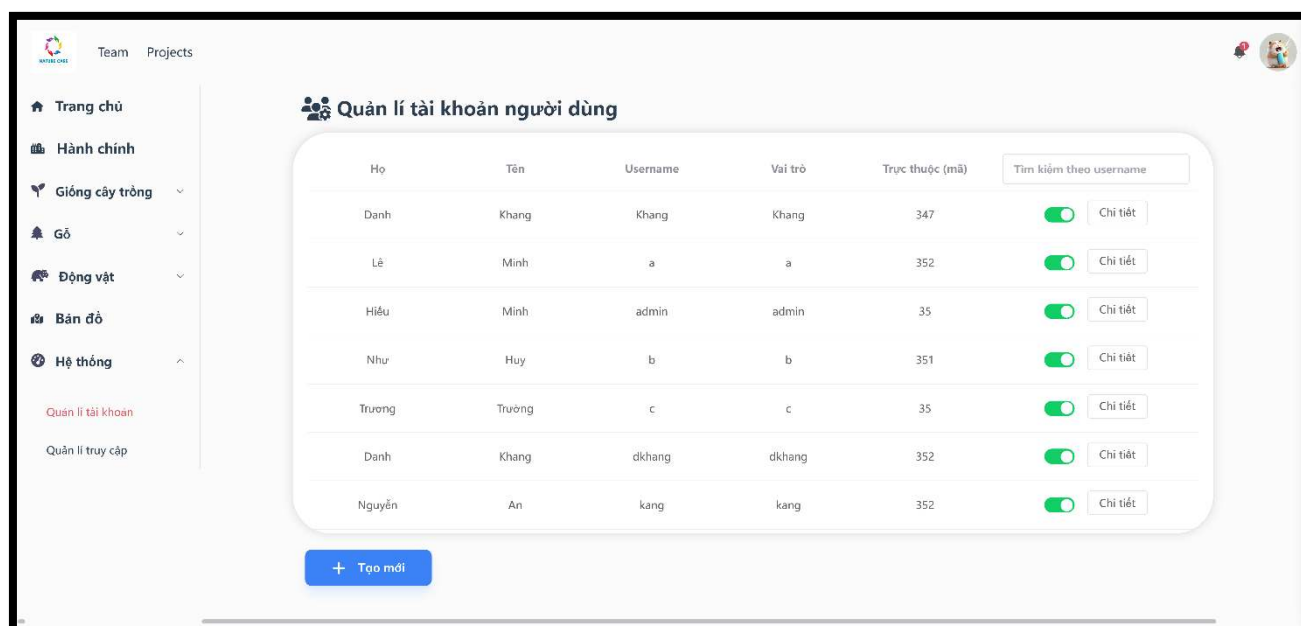


Hình 3.15- Giao diện Bản đồ phân bố của các cơ sở

3.3.7 Quản lý hệ thống



Hình 3.16: Giao diện tài khoản người dùng



Hình 3.17: Giao diện quản lý Users

3.4 Kết luận chung

Như vậy, ở phần 3 này, công việc đã được triển khai và hoàn thành. Kết quả cho ra được là một trang web với đầy đủ các tính năng như đã đưa ra ở pha phân tích thiết kế.

Kết luận

Kết luận chung

Trong môn học Kỹ thuật phần mềm, nhóm đã xây dựng được một hệ thống quản lý tài nguyên thiên nhiên. Thông qua việc nghiên cứu phân tích, phát triển và triển khai hệ thống, nhóm đã có kiến thức về các bước phân tích vấn đề; các pha thiết kế, triển khai hệ thống; có khả năng sử dụng những công cụ lập trình hiện nay như JavaScript, Java, HTML, CSS, Spring Framework. Ngoài ra là kỹ năng quản lý dự án mềm, teamwork, cách viết báo cáo, thuyết trình.

Hệ thống được xây dựng đã đáp ứng được hầu hết các yêu cầu đặt ra ban đầu. Hệ thống đã được đặt thử nghiệm tại địa chỉ: [tại đây](#) (Liên hệ với nhóm để được cấp tài khoản) . Tuy nhiên hệ thống còn một vài hạn chế như sau:

- Giao diện hệ thống còn đơn giản, chưa thể so sánh với những trang web được thực hiện bởi các công ty outsource có tiếng
- Còn thiếu một số tính năng nâng cao như tạo tài khoản bằng Gmail, Facebook

Hướng phát triển

- Bổ sung một số tính năng nâng cao hơn như đăng nhập bằng tài khoản Gmail, Facebook
- Tăng quy mô quản lý từ tỉnh Hà Nam đến quy mô quốc gia Việt Nam và Thế giới
- Tăng thêm những đặc tính quan trọng của các loài động vật, các giống cây trồng
- Tăng trải nghiệm người dùng thông qua cải thiện giao diện trở nên thân thiện hơn nữa

Tài liệu tham khảo:

- [1] Client-Server Model - GeeksforGeeks
- [2] Introduction to Java Programming, Comprehensive Version, 10E -
Y. Daniel Liang
- [3] Spring Framework Overview - Spring Framework
- [4] HTML là gì? - TopDev
- [5] CSS là gì? - TopDev
- [6] Introduction - Vue.js (vuejs.org)
- [7] What Is a Database - Oracle India